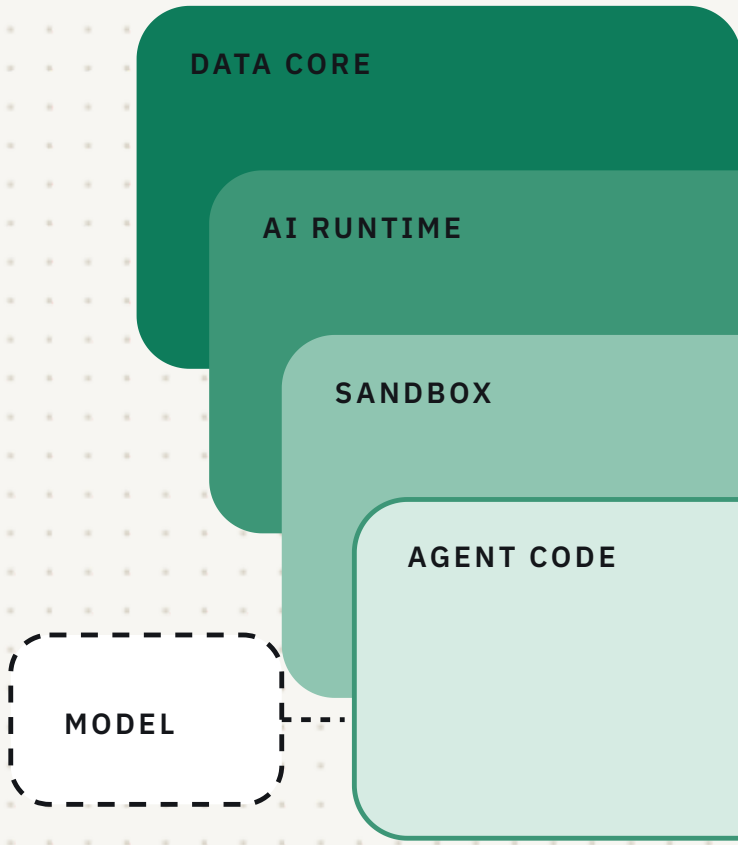
DecentAI



TECHNICAL PAPER · EDITION 1 · OCTOBER 2026

An operating system for AI agents, in which code decides and the model proposes.

The architecture, protocols and mechanisms of DecentAI: identity and policy, the gate every call passes, agents as declared contracts, confined execution, secrets that never reach the model, evidence, and how it is deployed and verified.

Ahmed Dirir · DecentAI · ahmed@decentai.io · decentai.io

ABSTRACT

The model may propose anything. It decides nothing.

Language models can now plan multi-step work and call tools, which makes them useful for acting on an organization's mail, documents, records and systems. They are also unreliable in two ways that matter for action: they state false things fluently, and they follow instructions embedded in whatever they read. The tools they call are programs written by third parties. A platform that lets such a planner act has to assume that the planner will sometimes be wrong or hostile, and that the code it drives may be hostile too.

DecentAI is a self-hosted platform built on that assumption. It separates *proposal* from *authority*. The model's output is treated as a request; every request is decided by code the model cannot read or influence: a deny-by-default policy walked on each call, a gate that compares a function's declared cost with the trust the person gave the chat and holds anything more expensive for that person's approval of the exact inputs, and an executor that runs approved agent code as its own operating-system user, inside a kernel-confined sandbox whose only way out is a proxy open to the hosts the agent declared. Credentials go from an encrypted vault to the agent's code for one call and never to the model. What the interface shows as data or as a finished step is drawn from recorded results, not from the model's prose.

This paper describes the whole system for engineers who will run, extend or audit it. Code paths, settings and limits are named as they are in the source; where a property depends on the host it runs on, the paper says so.

1 **Principles and threat model**
What is trusted, with what; the decisions every other chapter follows from.

2 **Architecture**
The engine's parts, containers and networks; where state lives.

3 **Anatomy of a turn**
One message, end to end, as a sequence of checked steps.

4 **Identity, tenancy and authorization**
Organizations, groups, policies, sessions, ownership and sharing.

5 **The assistant**
Context, the step loop, plans and verification, helpers, memory.

6 **The gate**
Permission levels, trust levels, approval cards and Safety rules.

7 **Agents**
The manifest as a contract; sources, review, approval and grants.

8 **Execution nodes and the sandbox**
Linking a node, workers, confinement, the proxy, the live screen.

9 **Secrets and data**
The vault, credentials, connected apps, records, files and storage.

10 **Evidence, monitoring and audit**
What a call leaves behind, and who can see it.

11 **Time and reach**
Schedules, webhooks in and out, notifications, stopping.

12 **Clients**
The web client, the phone app and the desktop app.

13 **Deployment**
One host, a separate node, two VMs on AWS, a desktop install.

14 **Verification**
Test suites, documentation checks, and the evaluation.

15 **Limits and open questions**
What the platform does not do or guarantee.

A **Glossary**
The vocabulary, in one place.

Sections are numbered within chapters (3.2), as are figures (Figure 3.2). Properties the system holds are stated once as numbered **invariants** (I-6.1) in a green margin, so that a reader can find them, test them and hold the code to them. Diagrams use one colour language throughout: ■ the platform's own parts, ▢ an agent's code, ⬚ the model, ▢ a step that waits for a person, ▢ refused or stopped. What the paper describes is the engine, its three clients, its deployment recipes and its catalogue of agents, as released in October 2026.

CHAPTER 1

Principles and threat model

Every mechanism in DecentAI follows from a short list of assumptions about what can go wrong and a shorter list of decisions about where authority lives. This chapter states both, so that the rest of the paper can be read as their consequences.

1.1 What is assumed to go wrong

The platform is designed against four kinds of failure, each assumed to happen in normal operation rather than as a rare attack.

WHO OR WHAT	HOW IT FAILS	WHAT THE DESIGN DOES ABOUT IT
The model	States false things fluently; follows instructions hidden in mail, pages and files it reads (prompt injection); proposes calls that are wrong, wasteful or harmful.	Holds no authority, no credential and no secret. Its output is a proposal that passes checks which do not read its prose (chapters 5–6).
An agent's code	Is written by a third party; may be buggy or hostile; may try to read other agents' files, reach hosts it did not declare, or escape its process.	Runs as its own user, confined by the kernel, with a proxy as its only way out, open to the hosts its manifest declared (chapters 7–8).
Content	A message, attachment, web page or tool result carries text written to steer the assistant.	Content is data. It can change what the model proposes, never what the platform permits.
A person	Approves without reading; holds more access than a task needs; leaves an organization.	Approvals show the exact inputs and bind to them; grants are narrow and revocable; ownership hands over when its holder leaves (chapters 4, 6).

Two parties are trusted: the operator who runs the deployment, who controls its hosts and keys, and the code of the platform itself. The paper does not claim protection against an operator with root on the host, nor against a compromised Data Core. It does aim to keep a compromise of any *other* part — the runtime, the sandbox, an agent, the model, a client — from becoming a compromise of the whole.

1.2 Where authority lives

The central decision is to put the model outside the chain of authority altogether. A conventional assistant lets the model's tool call *be* the action, gated at best by a confirmation dialog the model can describe in its own words. DecentAI treats the model's tool call as a proposal from an unprivileged party, and passes it to code that decides on facts the model does not control: the identity of the person, the policy of the organization, the agent's approved manifest, the chat's trust level and, where the call costs more than that trust, the person's yes to those exact inputs.

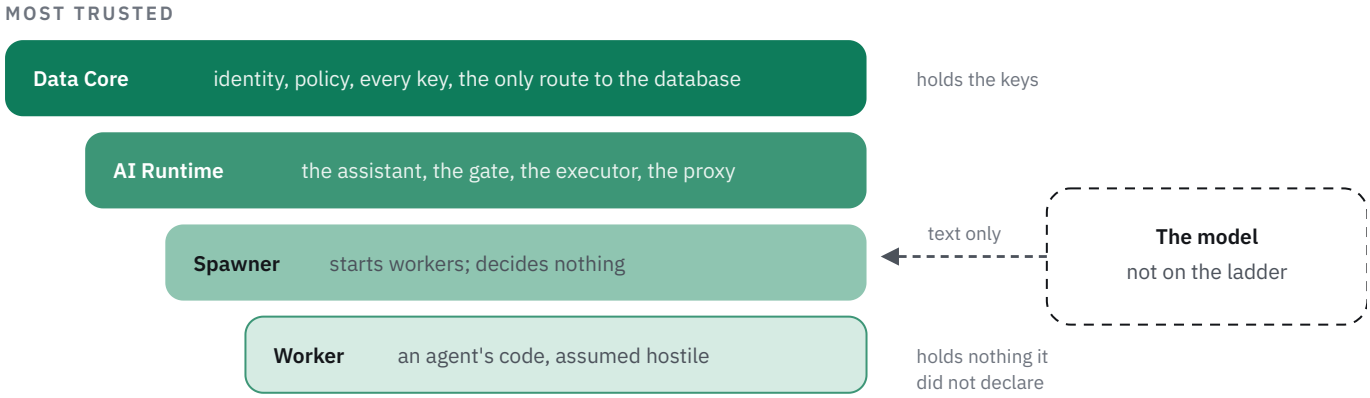


Figure 1.1 — The ladder of trust. Each part is trusted with what its job needs and nothing more, so taking one over gains as little as possible. The model is not a rung: it reaches the runtime as text, and nothing it writes is read as permission.

1.3 The decisions

Six decisions shape the implementation. Each is enforced in one place, so that it can be read, tested and argued about there.

Deny by default

No action is permitted unless a policy chain grants it to the person, walked on every request in the Data Core's one authorization point. A page that forgets to declare its action is refused, not opened.

One door

Clients, nodes and webhooks reach the engine through the Data Core alone. The runtime has no database and no route to one; it asks for what it keeps.

Contracts before code

An agent declares its functions, their cost, what it keeps and where it connects before anyone approves it. The executor holds it to that declaration on every call.

Decisions bound to inputs

A person's yes covers one call with one set of inputs, identified by a hash. Changing a character asks again.

Evidence over prose

Tables, files and "done" marks shown beside the model's words come from stored results of calls that ran. A claim with nothing behind it is shown as a claim.

Limits stated

Where a protection depends on the host — a kernel feature, a container runtime — the platform checks for it, refuses to start without the essential ones, and reports what is not enforced.

I-1.1 No model output is an authorization. A function runs only when the person's policy, the agent's approval and grant, the chat's enabled set and the gate all permit it, each decided by platform code from stored state.

I-1.2 No secret is ever placed in the model's context. Credentials travel from the vault to the agent's worker for one call (chapter 9).

I-1.3 The audit trail is append-only from the platform's side: no code path in the engine updates or deletes an audit row.

1.4 Why an operating system

The problem has a familiar shape. An operating system runs programs written by strangers, gives each a user of its own, mediates every request through a system call it may refuse, and keeps a log. DecentAI applies the same ideas one level up: an **agent** is a program with a declared manifest, a **worker** its process, a **function call** its system call, mediated by the gate; the **proxy** is its firewall, an **approval card** its "allow this app to…?" prompt, bound to one call;

the **clock** is the scheduler and the **audit trail** the system log. DecentAI is not a kernel: it runs on Linux in containers and borrows the kernel's own protections for the parts that must be confined. What it adds is a place for a planner that is capable and not trusted.

CHAPTER 2

Architecture

The engine is three programs — the Data Core, the AI Runtime and the Execution Sandbox — and the clients that speak to it. This chapter names them, shows what runs and what speaks to what, and says where every piece of state is kept.

2.1 The parts

PART	SERVICE	WHAT IT HOLDS AND DOES
Data Core	`data-core`	Identity and access, the agent lifecycle, the data layer, chats and approvals, organization settings, execution nodes and the clock. The only part that reaches the database; the engine's one published port.
AI Runtime	`ai-runtime`	One assistant per chat, the executor every call passes and its gates, what a schedule's fire runs, and the egress proxy agents reach the internet through.
Execution Sandbox	`sandbox`	Agent code: a spawner that starts workers, and one worker per call, each as the agent's own user.
Execution node	—	One AI Runtime with its sandboxes, deployed together. The bundled install has one; more can be linked from other machines (chapter 8).
Clients	released on their own	The web client, the phone app and the desktop app. Each is told the engine's address and speaks only to the Data Core, across origins, with a session token (chapter 12).

The three programs together are the **AI Engine**. The engine has no user interface of its own: every client reaches it over the same HTTP and WebSocket API. That separation lets one engine serve a browser, a phone and a desktop window, and lets each client be released on its own schedule.

2.2 What runs

The bundled install starts seven containers; six stay up and a seeder runs once at every start. Figure 2.1 shows them with the connections between them; every arrow is a connection its source opens.

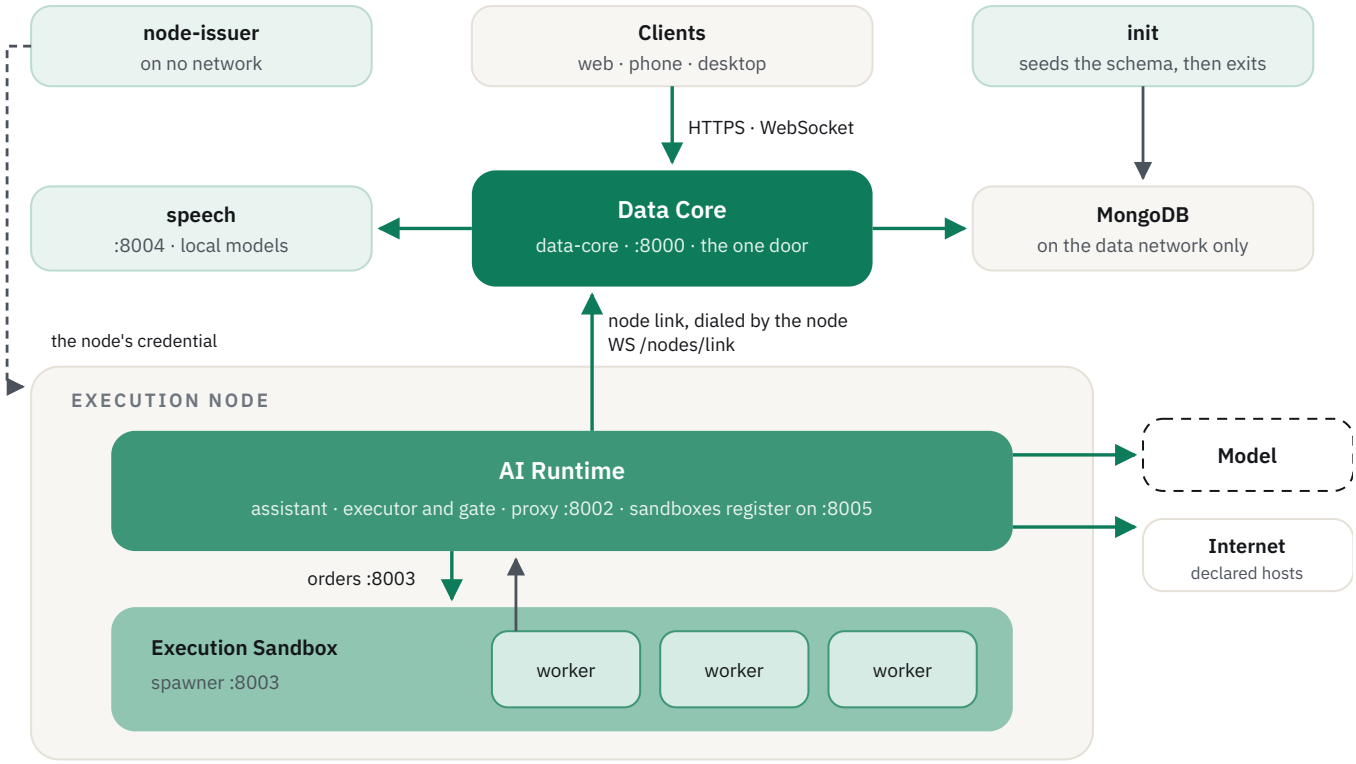


Figure 2.1 — The bundled install. The Data Core is the only published port and the only part that reaches the database. The execution node dials the Data Core's node link; the runtime orders the sandbox's spawner, and the sandbox registers with the runtime, each side proving the node's secret. Workers reach the internet only through the runtime's proxy, to the hosts their agent declared. The node issuer rewrites the bundled node's credential every few minutes from the install's authority key.

2.3 Networks

Three container networks carry the connections, and two of them are `internal: true`: they have no way out.

NETWORK	ON IT	PURPOSE
`default`	`data-core`, `ai-runtime`, `speech`	The one with a way out: clients reach the Data Core's published port; the Data Core reaches the speech service and the internet (repositories, mail, OAuth, outbound notifications); the runtime reaches the model's provider and, for agents, the internet through its proxy.
`data`	`mongo`, `data-core`, `init`	Internal. The database and the two that speak to it; nothing else has a route to it.
`sandbox`	`ai-runtime`, `sandbox`	Internal. The runtime is the only container on it with a way anywhere, so a worker's only way out is the proxy.

> **I-2.1** Only the Data Core holds a database credential or a route to the database. Everything the runtime keeps, it keeps by asking the Data Core, under the person's delegation for that chat.

> **I-2.2** An agent's worker has no network route except the runtime's proxy, and the proxy forwards only to hosts the agent's approved manifest declared.

2.4 Where state lives

The runtime and the sandbox are designed to be replaceable: a node can be restarted, drained or rebuilt without losing a chat. Durable state lives in the database and in a small number of volumes.

KEPT IN	WHAT
MongoDB	Every record of the platform: organizations, people, groups and policies; agents' approvals, grants and records; chats, messages, events and approvals; credentials (sealed); schedules, webhooks, notifications and the audit trail — 45 collections, all reached through org-scoped stores in the Data Core.
Uploads volume	Files, unless the organization stores them in S3, Azure Blob or Google Cloud Storage instead.
Agent packages	The approved code of each agent, by digest; nodes pull it from the Data Core.
Node volumes	Prepared agent environments, the sandbox's working folders and the node's credential: rebuildable from the above.
Speech models	Pinned model files, fetched once and checked against their published digests.

A chat in progress is held by one runtime at a time under a lease kept in the Data Core. If that runtime goes away, the lease lapses and the next one hydrates the session from the stored state — including a call parked for approval, which is saved before its card is shown (chapter 6).

Speech is the phone's or computer's own by default, or the organization's: a provider it named, or the platform's own models in the `speech` container (Whisper to write speech down, Piper voices to say answers). The container holds no key and no setting, speaks the providers' audio protocol so the choice between them is an address, and keeps nothing it is sent.

CHAPTER 3

Anatomy of a turn

One chat message, followed across every process boundary it crosses: the person's socket, the Data Core, an execution node, the model, the gate, a worker in a sandbox, and back. Each hop names the mechanism that carries it and the check that could stop it.

A turn is not a request and a response. A chat has one assistant with durable state, and a message is one more event delivered to it. The assistant then advances in **beats**: one model call, one action, one observation, one save. A turn ends when a beat says it is finished, when the work is blocked on a person, or when the turn budget runs out (`max_beats`, 20 by default). Figure 3.1 shows a turn with one call that needs approval; the sections follow its step numbers.

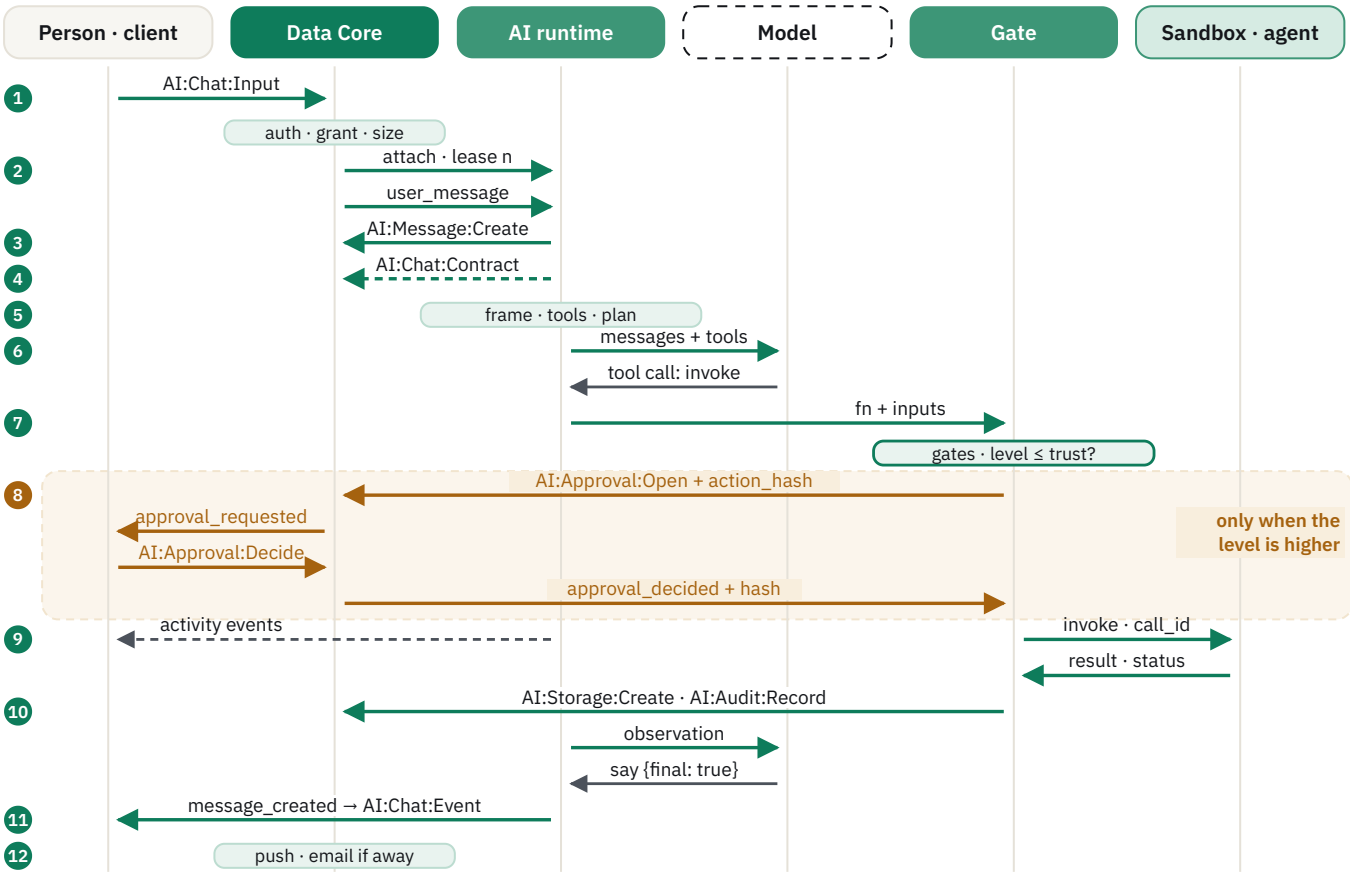


Figure 3.1 · One turn, one gated call. Green arrows carry the work, the dashed one is the contract read, and ink arrows come from the model or are relayed news. The amber band happens only when the function's level is above the chat's trust level. The Gate lane is the runtime's executor, drawn apart because nothing reaches the sandbox except through it.

3.1 The message enters the Data Core

Step 1. The web client keeps one socket per open chat, `WS /chats/{chat_id}`. A browser cannot set headers on a WebSocket handshake, and a token in the URL ends up in proxy logs, so the client offers its session token as a subprotocol: `Sec-WebSocket-Protocol: decentai.bearer, <token>`. A browser on the Data Core's own origin may use the session cookie instead, which counts only when the handshake's `Origin` is one of the app's own addresses (`PUBLIC_APP_URL`, `CORS_ALLOW_ORIGINS`). The socket closes with 4401 unless the caller is a person, with 4404 unless the chat is this person's in this organization, and with 4409 when the chat is archived.

Socket frames never pass through the `/app` gateway, so the socket checks each frame's action itself: `AI:Chat:Input` requires `ai:chat:sendmessage`. The text may be at most 256 KB, the page's message id at most 128 characters, and the attachments at most 20 file parts. A refusal is an `invalid_input` event and nothing is stored, so the transcript never holds words the assistant was not given.

3.2 Placement, lease and delegation

Step 2. Before forwarding, the Data Core's relay makes sure an execution node holds the chat. A person who stopped all of their work gets `work_stopped`. Otherwise placement picks the node holding the chat's live lease. Failing that, it picks the organization's own nodes first, then the node that last held the chat, then the least loaded; it never picks another organization's node. The relay then issues two things:

Lease	A record of node, number and expiry. The number rises every time the chat is given to a node anew. Heartbeats renew it every 20 seconds for 60. Every call for the chat carries `X-DecentAI-Lease`, and the gateway accepts only the live number from its node (otherwise 409 `stale_lease`), so two nodes can never both act for one chat.
Delegation	An HS256 token of type `RUNTIME`, naming a runtime-session record for this person, this chat and this node. It lives one hour and is renewed after 45 minutes by a `credential` frame. It is accepted only beside that node's own credential (`X-DecentAI-Node`).

Both travel in a signed `attach {chat_id, lease, credential, watched}` frame on the link the node dialled (`WS /nodes/link`); `user_message` follows. The delegation says *who*, never *what*. A node runs agents' code, so it holds nothing that outlives the chat.

3.3 The session hears the message

Step 3. On the node, the session host finds the chat's live session or builds one by hydrating the persisted assistant state. The session then works in a fixed order. It stores the message (`AI:Message:Create`); a resend with the same `client_message_id` finds the first copy and stops. It emits `message_created`, runs the pre-thinking hook, and records the inbox event (`AI:Event:Record`, which returns a per-chat sequence). Only then does it hand the event to the mind. A process that dies mid-beat loses nothing: the next one replays the inbox from its cursor.

Step 4. The hook re-reads the chat's **contract** (`AI:Chat:Contract`) and the model key (`Settings:Llm:Use`). The contract is the only message between Data Core and runtime that carries authority. It holds:

- the agents: installed ∩ granted ∩ enabled in this chat, each pinned by package digest and manifest hash;
- the function grants and the trust level (0–3, standard 1);
- the model, the turn budget, skills, routing and safety lists;
- the organization's own rules and the person's MCP servers.

Because the contract is re-read every turn, a grant withdrawn this morning is gone on the next message, and nothing has to be pushed to the runtime. An agent the node lacks is fetched by digest, and the audience sees `agent_status` while it installs.

3.4 What the model is shown

Step 5. The runtime renders the system frame with the parts that change least often first, because a provider's prompt cache keeps the front of a request (Figure 3.2). The tools are the 20 action tools plus one tool per function of each *opened* agent. Each function tool carries the manifest's input schema whole, and the total is capped at 128, the most OpenAI-compatible providers accept. The schema steers the model; the executor still validates every input.

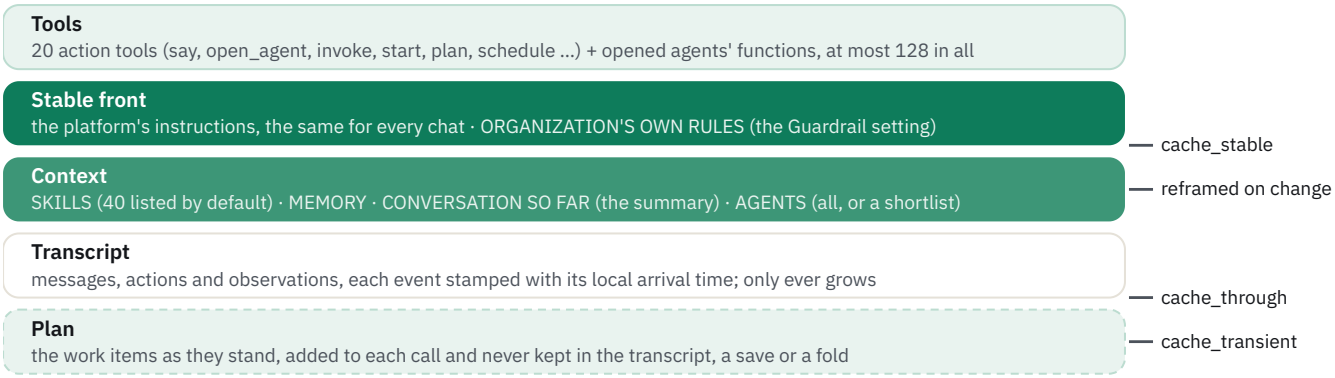


Figure 3.2 · The request one beat sends, front to end. Hints mark the end of the stable front, the last transcript message and the transient plan. The Anthropic, Vertex and Bedrock connectors turn the hints into cache marks; OpenAI and Gemini cache repeated prefixes on their own. No hint is ever sent to a provider as such.

Up to the routing threshold (15 agents by default), AGENTS lists every agent in one line. Past it, the router embeds the person's words with the organization's embedding model and keeps 50 candidates. It may rerank them with the chat's model, and it lists 15 plus those already open. A skill's body is loaded only on `use_skill`. Long transcripts are folded into the summary, and folded lines stay searchable through `recall`.

3.5 One model call, one action

Step 6. Each beat makes one model call and takes exactly one action back, as a tool call or as strict JSON text. A call to a function's own tool is rewritten to `invoke {function, inputs}` before anything reads it, so gate, trace and evidence share one vocabulary. A reply that does not parse is bounced, and one cut off at the output cap is never acted on. The model key is held in memory only while used and never reaches agent code.

3.6 The gate

Step 7. Every proposal, foreground or background, passes the executor's gates. They run in a fixed order, and none of them reads anything the model wrote around the call:

1. the function is declared in the approved manifest;
2. the chat's grants reach it; absence is refusal;
3. reference inputs resolve to stored results, and declared defaults are applied;
4. the inputs validate against the manifest's schema (NaN and Infinity are refused);
5. the scopes the call acts on are named as single values and fall within the grant's constraints;
6. the function's `permission_level` is at or below the chat's trust level.

Step 8. When the level is higher, the call is parked durably before its card goes out. `AI:Approval:Open` records the agent, function, resolved inputs, both levels, the job and the **action hash**: SHA-256 over compact, key-sorted JSON of `{agent_id, function, inputs}`. The page hears `approval_requested`. Only a person decides: `AI:Approval:Decide` is outside the delegation's fence, and the Data Core refuses it from a runtime with "A runtime never decides its own escalation." The decision is recorded first, with `approval.resolved` on the audit trail. Then the relay sends `approval_decided` carrying the hash the Data Core stored.

Before a yes is delivered, the contract is read again. A grant narrowed while the card waited therefore refuses the call. The level is not judged again, since the yes was to the call as shown. A call rebuilt after a restart must hash to the stored value, or it stops: "Stored action does not match its approval; execution blocked." A Deny also refuses the hosts the call named to every function of that agent for the rest of the request, so a function that needs no card cannot make the visit instead.

> **I-3.1** No call runs above the chat's trust level unless a person approved that exact `(agent, function, inputs)` triple. Its hash is recomputed before dispatch, and the grants are re-read after the approval.

3.7 Dispatch to a worker

Step 9. Past the gates, the executor builds a **call object**, the invocation's whole authority. It holds access to exactly the resource operations the function declared, and to a model only if the manifest says `llm: true`. The timeout is the manifest's `timeout_seconds`, or 60 seconds. The worker pool finds or starts the agent's worker in an Execution Sandbox that has registered and proved its locks; the runtime reaches it over TCP in TLS, authenticated by the node secret. The spawner starts the worker in the agent's own environment, as the agent's own user, fenced into its own files, with the egress proxy as its only route out. The call is one line of JSON (at most 2 MiB): `invoke {call_id, function, inputs, conversation}`. The worker holds no credentials. Each resource it needs is a request back naming its `call_id`, refused unless the function declared it. The host decides; the worker can only ask.

3.8 Result, evidence and activity

Step 10. The worker's `{result, status}` is a claim, not a fact. The host strips the keys only the platform writes (`storage_ref`, `displays`, `denied`, `not_permitted`). It validates a success against the output schema, stores it (`AI:Storage:Create`), and writes one `execution` line to the audit trail (`AI:Audit:Record`). Every call gets that line, whatever the outcome. Meanwhile the audience hears `activity` events (`call_started`, `agent_progress`, `call_finished`) sharing one `call_id`.

The mind sees the result whole up to about 16,000 characters, or as a 12,000-character preview with its `storage_ref`. **Evidence** comes from the trace, never from prose. A table or chart appears only when `say.show` names a successful traced call, and its rows are rendered from storage. Writes (level > 0) ride the message as `success` parts.

3.9 The answer, and the person who is not looking

Step 11. The next beat usually ends in `say {text, final: true}`. The runtime stores the message (`AI:Message:Create`, whose parts the Data Core validates) and emits `message_created`. The emission goes first to the durable log (`AI:Event:Append`), which assigns its `seq`, then up the link, and the relay hands it to the page as `AI:Chat:Event`. Answers are not streamed token by token: a turn streams as events from `working` to `idle`, the frame that ends a wait. A reconnecting page reads `hello` for the present and replays the rest with `AI:Event:List`.

Step 12. When a card opens or the assistant answers while no browser of the person has the chat open, the Data Core's notifier pushes to every subscribed device: Web Push with the deployment's VAPID pair for browsers, Expo for phones. It emails only if no device was reached, at most once per chat every ten minutes, and pushes answers at most every two minutes. The notification names the agent and step as the manifest does. Where the person or the organization asks, it says only that something happened.

> **I-3.2** Every event an audience hears that has a replay was recorded before it was sent. A socket is an audience, not a lifeline: detaching stops the live relay, never the work.

Events the client receives

Each arrives as `{"endpoint": "AI:Chat:Event", "data": {"event": …}}`. The list is defined once in the chat contract, which both the runtime and the Data Core use, and a test fails when the page's copy disagrees with it.

EVENT	MEANING	RECORDED
`hello`	on attach: protocol 2, `working`, `sleeping`, active jobs, pending cards, plan	socket only
`working` · `idle`	a cycle starts advancing · comes to rest	yes
`message_created`	a stored message: the person's echo, the assistant's answer, an agent's post	yes
`activity`	`call_started`, `call_finished`, `agent_progress`, `job_started`, `job_finished`, `helper_spawned`, `helper_said`	yes
`plan_updated` · `memory_saved`	the work items changed · a fact was remembered	yes
`approval_requested`	a call waits for the person: agent, function, level, inputs	yes
`question_asked` · `question_closed`	a card asks a question (words, files, code, a login, a skill) · it ended: `answered` or `expired`	yes
`schedule_set` · `schedule_removed` · `sleeping`	the clock changed; the assistant paused until a time	yes
`stopped`	the kill switch ended the work: counts of `jobs`, `children`, `cards`	yes
`agent_status`	an agent being pulled or installed: `pulling`, `installing`, `ready`, `failed`	socket only
`screen_frame` · `screen_closed`	a picture of a browser an agent drives (at most 300 KB) · it closed	socket only
`chat_titled` · `screen_unavailable`	delivered to an open chat, never recorded	no
`error`	the door could not take a frame; the socket lives on	socket only
`runtime_unavailable` · `runtime_disconnected` · `invalid_input` · `work_stopped`	added by the Data Core's relay: no node may serve the chat, the node went, the input was refused, the person stopped everything	relay

CHAPTER 4

Identity, tenancy and authorization

An identity is a row, never a bundle of permissions. What the row may do is found by walking a chain the organization keeps, on every request, and nothing is allowed unless something in that chain says so.

4.1 Organizations, users and groups

The **organization** is the top-level identity and the isolation boundary. Every user, group, role, policy, invitation, session and stored resource carries one `org_id`, and authorization never reads the mutable organization name. A **user** belongs to one organization. Their email is the login and is unique deployment-wide, so "who is this" never has two answers. A user's access is exactly their group membership. There is no administrator flag and no permission on a user directly, so "why may this person do that" is always a path that can be followed and changed.

Each organization has its own built-in **Everyone** group, which cannot be renamed or deleted. Its membership is implicit: the policy engine adds it whenever it resolves a user, so nobody can be missing from it. Standing an organization up seeds three chains:

POLICY	ROLE	GROUP	WHAT IT CARRIES
`FullAccess`	Administrator	Administrators	the wildcard, including every escape grant
`BaseAccess`	User	Everyone	the 69 baseline actions: one's own profile, memories, API keys, schedules, MCP servers, notifications. Chatting is not among them.
`MemberAccess`	Member	Members	37 everyday actions: `ai:chat:*` and the rest of one's own chats, one's own files, records and credentials, and seeing the agents one was given

The baseline is versioned. A new baseline action is offered to each policy once, because re-seeding on every restart would undo an administrator's deliberate removal.

4.2 Roles, policies and action strings

A **role** bundles policies and reaches a person only through a group. A **policy** holds at most 50 statements of at most 100 actions each:

```
{"effect": "Allow", "actions": ["agents:agent:install", "iam:group:*"], "resources": ["*"]}
```

An action is an endpoint. `POST /app {"endpoint": "Agents:Agent:Install"}` is checked as `agents:agent:install`, so there is no second permission vocabulary to keep in step with the API. The vocabulary is the **action catalog**: 224 actions in 43 services, 69 of them baseline. It is part of the code under review rather than a file a deployment can edit, because every entry is a security boundary. The policy editor is drawn from the catalog. Each saved pattern is validated against it: an unknown action, or a wildcard whose text before the `*` begins no real action, is refused. `resources` must be `["*"]`; who sees an individual document is a question for sharing (§4.7).

An installed agent's functions are deliberately *not* in the catalog. They come and go with manifests, and authored policies naming them would rot. They are granted by **agent grants** instead: allow-only rows keyed by the agent, each with an owner map of groups and users, a function list or `*`, and optional constraints. These are served as the contract's `grants`. A bare `*` in a policy grants no agent function at all.

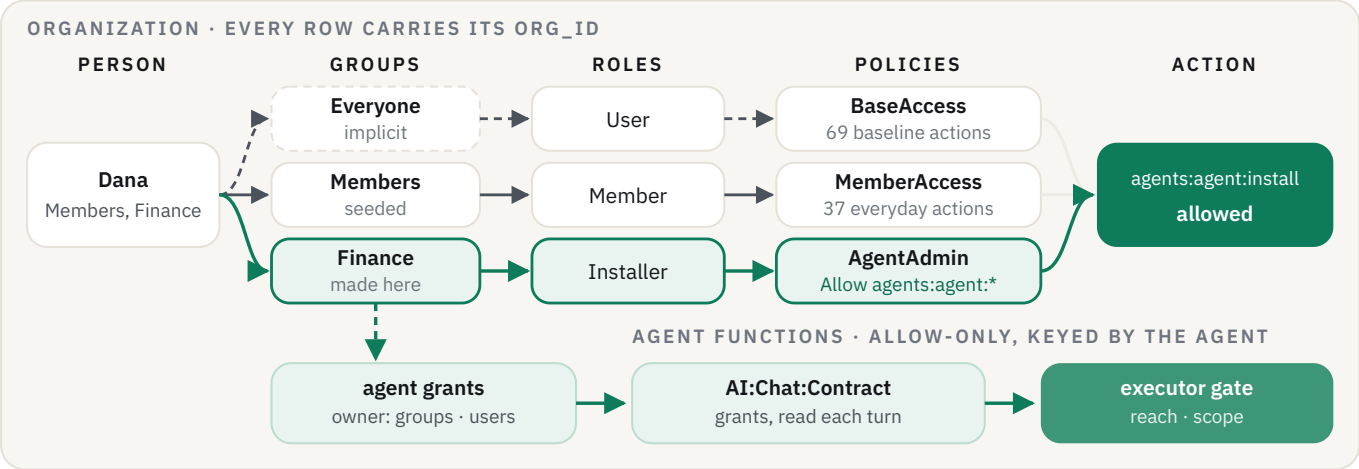


Figure 4.1 · How a grant reaches a person. Top: the IAM chain, walked on every request, which Everyone joins implicitly. Only the Finance path matches `agents:agent:install`, and a matching Deny anywhere in the chain would override it. Bottom: calling an installed agent's functions is a separate, allow-only grant naming groups or people, which reaches the gate only through the chat's contract.

4.3 One decision, in one place

Routes authenticate; the gateway's router authorizes, once for every endpoint, and controllers hold no authorization code. The router lowercases the name and refuses one starting with an underscore. It then asks the access controller, and only after that requires the action to be in the catalog, so a wildcard cannot reach a helper a controller happens to expose (Figure 4.2):

- A **node** may call the three package-door actions and nothing else; there is no person whose policy could apply.

- A **runtime** delegation is first held to the delegation's fixed list of 41 action patterns, before its person's policy is read. Even an administrator's delegation therefore cannot reach identity, account settings or approval decisions. The Data Core then checks that each chat-bound call names the delegation's own chat.
- For everyone, the policy engine takes every statement reachable through the user's groups, Everyone included. A matching `Deny` refuses at once; otherwise a matching `Allow` permits; no match refuses. Matching is case-exact and identical on every operating system.

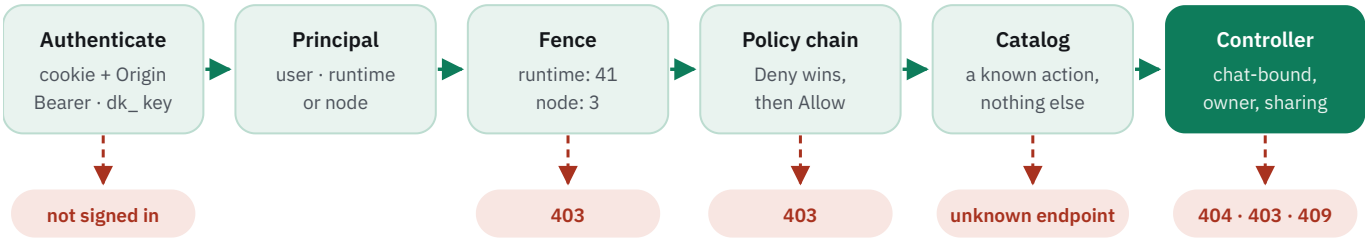


Figure 4.2 · The order of the checks on `POST /app`. The first check that fails is the answer. The controller's checks concern a record (is this chat the delegation's, may this person see or change this document); whether the caller may perform the action at all was settled before the controller is reached.

Resolution is cached per process for 60 seconds, but every write that could change a decision invalidates the cache before and after the write, and a resolution that began before an invalidation is not kept. A revocation made through the application therefore counts from the next request. One made by another process, from the command line, takes effect within the minute.

4.4 Tokens and sessions

A session token is a pointer, not a container. Signing in writes a session row and returns an HS256 token that *names* it. Its claims find the row and decide nothing. Every request resolves session and user again, from the database or from a copy kept at most a minute and dropped on change, so signing out, disabling and deleting take effect at once. Sessions last 7 days, or 90 days when the person asked to stay signed in.

CREDENTIAL	PRESENTED AS	NOTES
Browser session	`access_token` cookie	for a browser on the Data Core's own origin; accepted only from the app's own origins, because the browser sends the cookie to every port of a host
Web client session	`Authorization: Bearer …`, and on the chat socket the subprotocol `decentai.bearer`	signing in with `"client": "web"` returns `access_token` and `expires_at` in the body
Native app session	`Authorization: Bearer …`	`"client": "native"`; kept in the device's store; may list API keys but not make or revoke them
API key	`Bearer dk_…`	the person themselves, looked up on every call and never cached; only the hash is stored; 20 per person; made and revoked only from a browser session, so a leaked key cannot make itself permanent
Runtime delegation	`Bearer` + `X-DecentAI-Node` + `X-DecentAI-Lease`	one person, one chat, one node; one hour; held to the 41 runtime actions; survives sign-out (a scheduled chat runs while its person is away), but not disabling or deleting the person
Node credential	`Bearer` on `WS /nodes/link`	signed by a registered authority's private key (RS256, ES256, EdDSA); minutes long; opens only the package door

Passwords must be 10 to 256 characters with a letter and a digit. They are hashed with bcrypt over a SHA-256 pre-digest, so bcrypt's 72-byte limit never truncates a passphrase. Sign-ins are throttled in a 15-minute window: 5 attempts per account and 20 per address. The throttle is checked before the password, so a locked window answers

a right password exactly as it answers a wrong one. A reset link lives an hour, works once and is stored only as a fingerprint. Using it ends every session and API key the person had.

> **I-4.1** No authority is copied into a token. Every credential names a row or a principal, and what that principal may do is read from the live chain, or for a chat from the live contract, at the moment it acts.

4.5 First run, setup code and invitations

Nobody signs themselves into an existing organization. An organization is created in one of three ways, all through the same provisioner.

- **First run.** While the deployment has no users, `POST /auth/setup` (`org_name`, `name`, `email`, `password`) creates the organization with that person as administrator. A single first-run claim makes two simultaneous submissions produce one administrator; once anybody exists, the call answers 409. Where the address is reachable from other computers, setup also asks for a **setup code**. The deployment generates one and writes it to its log when `SETUP_CODE` is empty, uses the operator's value when one is set, and asks for none when it is `none`.
- **On the machine.** The operator's bootstrap command creates the organization and invites its first administrator.
- **Sign-up.** Only with `ORG_SIGNUP=open` and mail configured. Nothing is created until the emailed link is followed, and the answer is the same whether or not the address has an account.

Invitations store only the token's hash, one live per address, valid for seven days. The account's address and organization come from the invitation, never from the request, because the link proves one mailbox and nothing more. The invitation's groups are given only if the inviter could still give them at acceptance. Without mail, an administrator adds the person (`iam:user:create`) with a temporary password. Signing in with it answers `change_required` and opens no session until the person chooses their own.

4.6 Two invariants

> **I-4.2** An organization cannot lock itself out. Every change to the access chain checks, before it commits, that at least one *active* user of *this* organization still holds `iam:user:set_groups`, the recovery capability. If none would, the change undoes itself and is refused.

> **I-4.3** Nobody grants what they do not hold. An author may only *allow* actions they hold themselves. This is checked when a policy is written and again at every attachment: a policy to a role, a role to a group, a group to a user. Escalation therefore cannot be laundered through an attachment. A `Deny` can only narrow access, so anyone may write one.

4.7 Ownership and sharing

Who may see a document is its owner map, a set of groups and a set of users. **Who may change** it is its steward, the person recorded as its creator. One sharing engine serves every shareable type, because two copies would drift and the next shareable type would be the one that leaks. Stores filter every read through it, organization pin first, then owner map, so a document out of reach does not exist as far as the API can tell. Everything starts private. A writer shares only into groups they sit in and with their peers, and an id from another organization is refused rather than stored. Seeing is never authority: a colleague can use a shared credential but cannot edit or delete it.

DOMAIN	SHARING	STEWARD AND ESCAPE GRANT
Secrets, files, records, skills	personal: organization-wide only with the escape grant	the creator; `…:set_owner_any` per domain, reaching only documents its holder can already see
LLM connections, agent sources	infrastructure: Everyone open to the creator, who always stays in the map	`settings:llm:manage_any`, `agents:agent:source_manage_any`, reaching every one in the organization
Chats, memories, MCP servers, API keys	never shared	the person only; no escape grant
Installed agents, agent and credential grants	the organization's	whoever holds the action (`agents:agent:delete`, `…:grant`)

An agent's use of a credential is a grant, not a share. It is a revocable record per agent and slot that rides on visibility, and it checks a connected account's provider and consented scopes. Stewardship is transferable (`secrets:secret:transfer` and its siblings). The new steward takes the old one's place in the owner map, so every viewer keeps seeing the document.

4.8 Hand-over, and isolation between tenants

Deleting a user is a hand-over. `IAM:User:Leaving` previews what the person owns, by kind. `IAM:User:Delete` names a successor, an active member who is the deleting administrator by default. In one pass it:

- drops sessions, keys, invitations and delegations;
- deletes chats with everything under them, memories, MCP servers and webhooks;
- passes stewardship of secrets, records, files, skills, model connections and agent sources to the successor, with every share intact;
- takes the person out of agent grants;
- keeps the audit trail as recorded.

Disabling keeps everything, but drops sessions, keys and delegations, closes live chats and pauses every schedule, so nothing acts unattended for someone who may not act.

Isolation lives in the queries, not in filtering afterwards. Organization-scoped stores put `org_id` beside every id a caller supplies, so another tenant's id finds nothing, and the visibility filter pins the organization before it reads an owner map. Nodes keep the same boundary. Placement never uses another organization's node. An organization may refuse shared nodes (the `own_only` setting of `Settings:Nodes`). A node admitted by one organization's authority is told only of that organization's packages. A disabled organization keeps its data and refuses everyone in it; the application has no way to delete an organization.

CHAPTER 5

The assistant

Each chat has one assistant. It is a durable record, not a process. A session loads it, feeds it events, lets the model choose one action at a time, and saves it after every step. The model chooses; the platform checks each choice, carries it out, and records only what actually happened.

5.1 One mind per chat, kept as state

There are no turns that start from nothing. A user message is one more event reaching a mind that already exists. That mind is one serializable state object: the working transcript, a summary and archive of what folds removed, the plan, the opened agents, the *trace* of every invocation with inputs, status and result, the jobs table, the one foreground call parked on a person, the inbox `cursor`, the beat count and a `stopped` flag. It is saved whole on every step (`AI:State:Save`). Any process can load it and continue: the watched session, a headless one, a fresh process after a deploy, another node after a failure. A failed save costs one step, not the work.

5.2 What the model is shown

Each step sends one request, rebuilt from state and ordered by how rarely each part changes, because a provider's prompt cache keeps the front of a request:

PART	WHAT IT HOLDS
Tools	The actions as tool schemas, plus every function of an *opened* agent with its manifest input schema whole; at most 128, the rest callable by name. Tools are only ever appended.
Instructions, rules	The platform's instructions for every chat, then the organization's own rules (§6.6). Together, the stable front of the frame.
Skills, memory, summary	One line per skill (40 by default), the person's remembered facts, and what folds took out of the transcript.
Agents	One line per enabled agent; past 15 agents, the open ones plus an embedding shortlist of 15 for the latest message.
Transcript, plan	Events and observations, each stamped with local time; then the plan, added to every call and never kept.

Details cost space only once asked for. `open_agent` loads an agent's catalogue with each function's price, marked `NEEDS APPROVAL` where it exceeds the chat's trust level, so the model knows before it chooses. Invoking an unopened agent is refused, since it means guessing inputs. At most eight agents stay open. A result over 16,000 characters enters as a preview of whole items plus its `storage_ref`, which `read` pages through.

5.3 The step loop

One *beat*: absorb events, think once, act once, observe, save. The model returns exactly one action from a fixed vocabulary of twenty (`say`, `open_agent`, `invoke`, `start`, `spawn`, `plan`, `remember`, `schedule`, `sleep`, `finish` and their kin). A call to a function's own tool is rewritten to `invoke` first, so the gates, the trace and the evidence share one vocabulary.

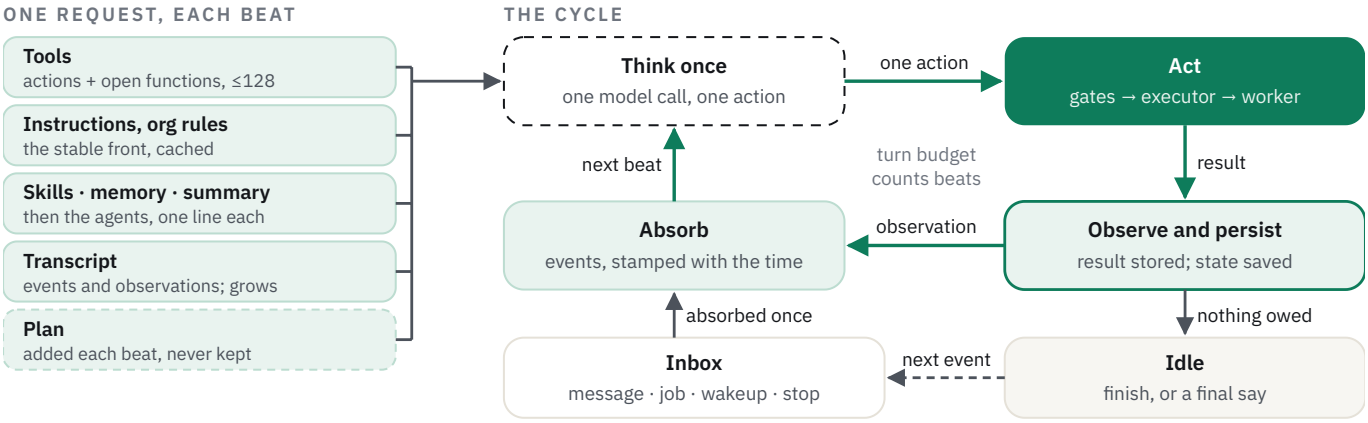


Figure 5.1 The step loop. The request is rebuilt from state each beat; the dashed box, the model, only proposes. Every action passes the gates before anything runs, and the state is saved before the next beat.

Malformed replies are corrected, not acted on: an invalid action bounces (two in a row cost no beat), prose with no action is delivered as a `say`, a reply gluing several actions runs only the first, a `say` 92% alike to one already sent is refused, and a reply cut off at the output cap is never acted on. A `finish` carries a reason checked against the state: `completed` is refused while plan items are owed, `awaiting_events` when nothing runs or is scheduled, `blocked` when nothing is blocked.

The turn budget is a valve against runaway loops, not a quota. It counts beats since the person last spoke, up to the chat's `max_turns` (20 by default; 0 means none). At the limit the model is told to wrap up honestly and gets three more beats; then the runtime pauses the work, intact and resumable. A helper that runs out reports `budget`, not "done".

5.4 The plan, and nothing invented

The plan is the assistant's to-do list, shown to the person: at most 12 items of 200 characters, each `pending`, `active`, `done` or `blocked`, with runtime-assigned ids. A blocked item needs a blocker; an item becomes active only when its dependencies are done. While an item is active, every successful call's storage ref and every finished job's id land on it as evidence. The model may name evidence too, but only refs the trace holds. Done without evidence is kept and shown *unverified*, since "answer the question" leaves no trace. The rules are enforced in code; the prompt only describes them.

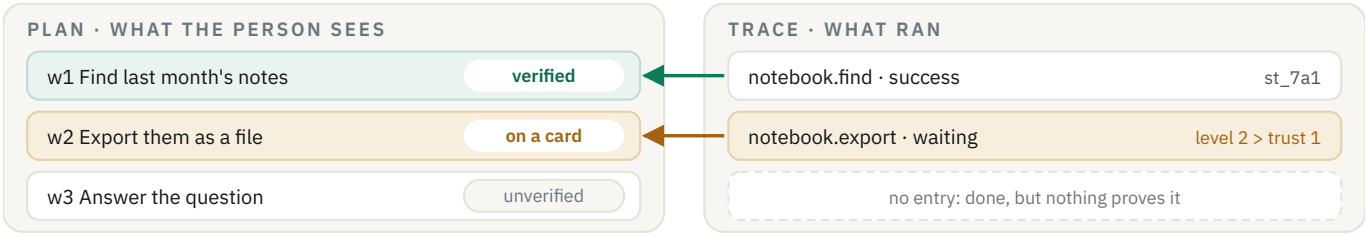


Figure 5.2 The plan and the trace. The words of each item are the model's; what marks one verified is a trace entry produced while it was active. A `finish` claiming completion while w2 is owed is refused, and so is evidence naming a ref the trace does not hold.

> **I-5.1** A plan item is verified only by refs the trace produced. A `finish` that claims completion is refused while any item is pending or active.

No prompting stops a model from inventing things, so the platform guarantees something narrower: *anything shown as data was read from a real result*. A reply has two voices. The words are the model's. Everything beside them is assembled by the runtime's evidence step from successful trace entries, never from parsed prose: a table or chart the `say` names (up to three), only if a successful call stored those rows or offered that display, rendered from storage so no value is retyped; a `success` part for every write above level 0; files only from calls that produced them. A show it cannot back is dropped and the reason goes to the model. Keys only the platform may write (`storage_ref`, `displays`, `denied`) are stripped from worker output, which is validated against the declared outputs before it is stored.

> **I-5.2** Every table, chart, file and "verified" mark on a message comes from a successful trace entry and its stored result. A claim with nothing behind it is shown as the model's words alone.

5.5 Jobs and helpers

`invoke` holds its beat until the call answers; `start` runs the same call as a *job* whose result arrives as a `job_done` event, through the same gates and into the same trace. A job whose process died is settled as an honest error, never retried on a guess. `spawn` hands a bounded goal to a *helper*, a job whose worker is a session. Paraphrased context is where invented values come from, so the goal is the whole brief: the helper never sees the parent's transcript, summary or plan. It runs under the parent's contract, optionally narrowed to fewer agents. At most three are live per parent, and depth is one by construction: a helper cannot spawn, sleep, schedule, remember or save a skill. Its trace entries join the parent's, so its work is presented through evidence. Its job is `done` only if it finished `completed`, and success with no successful call leaves its items blocked.

5.6 Memory, skills, MCP servers, models

Memory. `remember` saves one durable fact, visibly. The runtime may list and add memories but holds no verb to delete one: a model that could forget could quietly rewrite what it knows about someone. **Skills** are Markdown instructions, listed in the frame and loaded by `use_skill`. A skill outlives its chat, which makes it what an injected page would most like to plant, so only the person asks for one; `save_skill` opens a card showing it whole, and the save is the person's own click, made as them. **MCP servers** a person adds appear beside the agents as `<server>.tools.<id>`. A server does not say what its tools do, so each tool starts at level 3; a tool that is new or changed comes back switched off; address and credential are fetched only at call time (`Mcp:Server:Use`). **Models.** A connection is a provider and a key; the model is chosen per chat, and the key travels as a `secret_ref`. Connectors cover OpenAI-compatible, OpenAI Responses, Anthropic, Bedrock, Gemini and Vertex AI. When a chat's model changes, the transcript says so once, and the new model is told the earlier actions were not its own.

5.7 Steering, stopping, sleeping

The person may write at any moment. The mind gives way at once: a reply still being written is dropped and asked for again, uncounted; a foreground call continues in the background as a job; a call showing a screen receives the words too.

CONTROL	EFFECT
Stop generating	Honoured between beats; a call the beat waits on ends at once; jobs are cancelled and given 15 seconds; the stop is written into transcript and state, so a reloaded mind rests.
Stop everything in this chat	The kill: pending cards are expired on the record first, then the cycle, jobs, workers, helpers, browser and sleep end where they stand.
Stop all my work	`AI:Switch` : nothing of the person's is placed on a node, every chat is killed with ten seconds to record itself, and every runtime key for them is withdrawn. It holds until they resume.
`sleep`	A pause of at most a day while waiting on something outside, one per chat; a message is still heard at once, and a stop cancels it.

5.8 The lease, and hydration after a restart

Two sessions never advance one assistant. Within a node, a registry and a per-chat build lock hold one session per chat. Across nodes, a numbered lease holds each active chat on one node, renewed by heartbeat every 20 seconds for 60. Every call a node makes for a chat carries the number (`X-DecentAI-Lease`); the gateway takes only the live one and answers others `409 stale_lease`. A lapsed lease is re-placed under the next number, never sooner, so a slow node never finds another acting for its chat. Hydration then rewrites the frame from the present, settles jobs left `running` as unknown (a helper resumes from its own state), expires questions a dead process left, absorbs every inbox event past `cursor` exactly once, and resumes the cycle if the mind was left mid-work and not stopped. Approval cards stay open: their parked calls are durable (§6.3).

I-5.3 At most one process acts for a chat at a time. A call that was running when its process died is reported as unknown and never re-run on its own.

CHAPTER 6

The gate

Every call the model proposes passes one chokepoint, the runtime's function executor. It compares a price on the function with a trust level on the chat. When the price is higher, the call waits for a person. The model is never an input.

6.1 Two numbers and one rule

Every manifest function declares `permission_level` 0–3: a read, an ordinary change, a wider change still inside the platform, an action that leaves it. The level is a price, not a permission; a reviewer reads it beside the function before approving the agent. Each chat has a *trust level* on the same scale, chosen by the person: *Ask for every change*, *Standard*, *Trusted*, *Autonomous*. The rule is one comparison: a function runs when its level is at or below the chat's trust, and asks first when above. Which functions exist for a person is decided by grants; trust decides only how far work goes unasked. The default is 1, and a stored value that is not 0–3 reads as 1, never 3, so damaged data never resolves to the most permissive setting.

FUNCTION LEVEL	0 · ASK FOR EVERY CHANGE	1 · STANDARD	2 · TRUSTED	3 · AUTONOMOUS
0 · read	RUNS	RUNS	RUNS	RUNS
1 · ordinary change	ASKS	RUNS	RUNS	RUNS
2 · wider change	ASKS	ASKS	RUNS	RUNS
3 · leaves the platform	ASKS	ASKS	ASKS	RUNS
Not granted, out of scope, invalid inputs, blocked site, unlisted package, host denied this request	REFUSED	REFUSED	REFUSED	REFUSED

Figure 6.1 The matrix: the function's declared level against the chat's trust level. *Asks* parks the call on an approval card with its exact inputs. The last row is independent of trust: refused before any card, with the reason told to the model.

6.2 The order of the checks

In fixed order, each failure returned to the model as an observation: the function is declared in the approved manifest; the grants reach it (deny by default); references are resolved to stored values, defaults applied, and inputs validated against the manifest schema; scopes are read from the inputs the manifest binds them to and held to the grants, each named as one value; the level is compared with trust; the call runs in its worker under its timeout, and its output is validated, stored and audited whatever the outcome. The chat's contract (agents, grants, trust, model, Safety) is read again before every turn, so access taken away this morning is gone this turn.

6.3 The approval card

A card shows the agent, the function, its level and the *exact inputs*, after defaults and references, so the person approves values, not a ticket. An *action hash*, SHA-256 over sorted compact JSON of `(agent_id, function, inputs)`, is stored on the card and kept with the parked call. The park is durable before the card exists: the runtime writes the parked call, or the job set to `waiting_approval`, into the state and saves it, and only then emits `approval_requested`. If that save fails, the card is withdrawn and the call reports that nobody could be asked, never that someone refused.

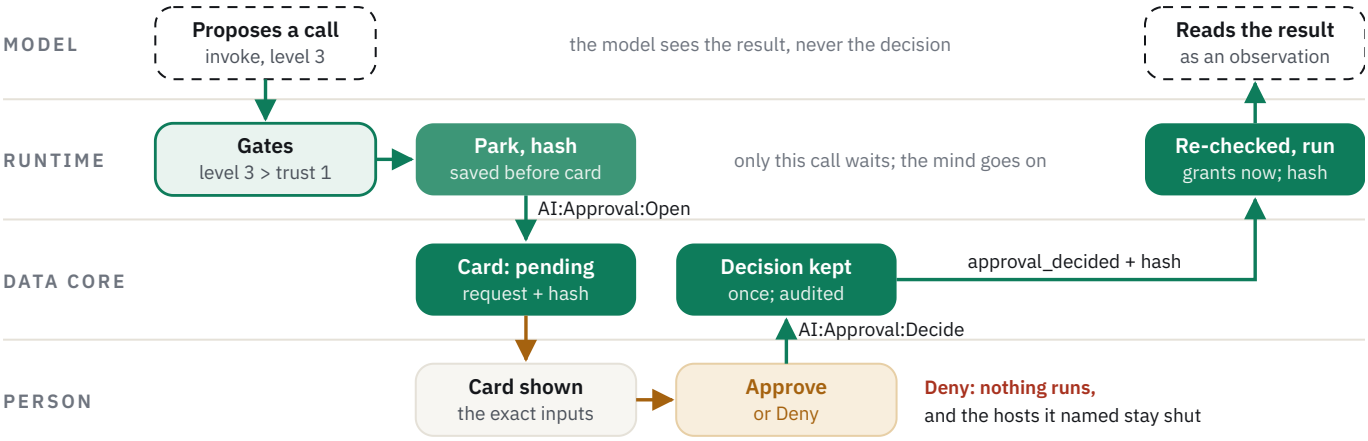


Figure 6.2 The life of an approval, left to right. The call is parked and saved before the card exists, so a decision days later still resumes it. The Data Core records the decision before sending it; the runtime re-reads the contract, checks grants as they are now and compares the hash before anything runs. Amber steps wait for a person.

Only a person decides. `AI:Approval:Decide` is a person's verb; a runtime principal calling it hears "A runtime never decides its own escalation". The decision is recorded first, audited as `approval.resolved`, then sent. It is a conditional update that lands only while the card is `pending`, so a second decision gets `409 not_pending` and never overwrites the first. The runtime ignores a decision for a card its chat does not hold. An approval has no deadline: a parked job waits while the mind works on, and a foreground `invoke` holds its beat until the card is answered or the person writes. On a yes, the level is not judged again, since the yes is to the call as shown, but the grants are: an entitlement narrowed while the card waited must not be carried through.

IF	THEN
the person denies	Nothing runs. The hosts the call named are refused to every function of that agent, at any level, until the person's next message, so a function that needs no card cannot make the visit anyway.
the process died meanwhile	The card stays open; the decision resumes the call from the saved park, every gate re-run and the hash recomputed. A mismatch blocks execution.
a grant was narrowed	The yes is refused `not_permitted`.
the call was stopped or killed	The card is expired on the record and closed on the page.

> **I-6.1** What runs after an approval is exactly the call the card showed: its inputs hash to the value recorded when the card opened, and it passes the grants as they stand when the yes arrives.

> **I-6.2** A decision is recorded by the Data Core before it reaches the runtime, lands once, and can only be made by a person.

6.4 Questions, credentials and code

The same record, `ai_approvals`, carries every card where the platform needs an answer it cannot know. These are `kind: question`, told apart by `expects`, and wait at most a day, the function's own timeout paused meanwhile. A question lives only as long as the call that asked it.

CARD	SHOWS	ANSWERED WITH
Question	an agent's question, up to 8 choices	a choice, words, or a file
Files	up to five files the person can see	refs ticked, each re-checked against what they may see
Credential	entry, consent, choose or once; host, site, field labels only	values go straight to `Secrets:Credential:Save`, never to the card's record
Code	the code whole, purpose, packages, hosts, credentials, files, review	allow or deny; allow opens the named hosts for that call only
Skill	title, summary, body as plain text	save or decline; a save creates it as the person

Code proposals exist because the Browser runs scripts and the Code agent writes programs that nobody can list in advance. A package not on the organization's list is refused before anyone is asked. Then the chat's model reads the code and the card carries its verdict: `agrees`, `differs` or `unread`; code that would break the organization's rules never agrees. The review advises and never decides, and one sent by an agent is discarded. An organization may let some code through without a card, but that is decided in the Data Core as the card opens, so the runtime cannot loosen itself. Only code the review agrees with qualifies, it is still announced in the chat with its first 25 lines, and every looser choice rests on an earlier yes from a person.

6.5 The organization's Safety rules

Settings → Safety is the administrator's, for everyone, and each row is enforced in code: **sites no agent may open** at the proxy, for every worker and every manifest, bare addresses refused while the list holds names; **scripts and programs** in the Data Core as the card opens; **packages** in the runtime before any card; **MCP servers and inbound webhooks** in the Data Core (blocked webhooks answer 404); and **what a notification says** in the notifier. Notifications pass through services outside the deployment, so the hidden setting reduces every push and email to "Something is waiting for your decision" or "A chat of yours has an answer", with a link; a person may choose the same but cannot undo the organization's choice. Below the rows sit the organization's own rules for the assistant (`Settings:Guardrail`, at most 4,000 characters), placed in the frame and the code reviewer's instructions. They are guidance, not enforcement: no code reads them. What must never happen belongs in the rows, the grants and the trust levels, which hold whatever any prompt says.

6.6 Why the model decides nothing

A model's output is shaped by everything it read, including pages, emails and results written by people who are not the user. So nothing that grants authority reads it. The level comes from the reviewed manifest, trust from the chat's settings, grants from policy served in the contract, and a yes from a person through a verb no runtime can call. The runtime acts under a one-chat delegation that lasts an hour and opens only a fixed list of runtime verbs: it can open and expire cards but not decide them, add memories but not delete them, read skills but not write them. The model may propose anything; a refusal reaches it as an observation it can explain, and nothing it writes afterwards reaches a parked call's inputs.

> **I-6.3** No input to the gate is produced by the model. Levels, trust, grants and decisions come from the manifest, the chat's settings, policy and a person. The runtime can open a card but never answer one.

CHAPTER 7

Agents

An agent is a folder in a git repository: a manifest that declares everything it may do, keep and reach, and Python that does it. The platform reads the manifest without running the code, an administrator approves one exact version of both, and from then on every call is held to the stored, approved manifest — never to anything the running code says about itself.

7.1 An agent is a package

Nothing about an agent is registered elsewhere or found by convention. The folder holds the manifest, which is the contract; the agent's entry point, which returns its tools; one class per tool, with one asynchronous method per function; a README written for the reviewer; and, optionally, sample records and files a person can load in one click. A function returns a result and a status, success or error; anything else, an exception included, becomes an error result.

At approval the folder is packed canonically: files sorted by POSIX path, mtime and owner zero, mode 0644 or 0755, PAX tar, gzip level 9 with a fixed header. The **package digest** is `sha256:` over the gzipped bytes, so one commit always gives one digest and any changed byte gives another. A package holds at most 5,000 files and 64 MB uncompressed, and no symbolic link.

7.2 The manifest

The manifest has seven top-level blocks: `schema_version` (exactly `"1.0"`), `agent`, `implementation`, `network`, the optional `authorization` and `resources`, and `tools`. Figure 7.1 is the Gmail agent's, condensed to the lines that carry the design.

```yaml
schema_version: "1.0"                  # the one grammar; any other value is refused
agent:
  id: gmail                            # permanent: first segment of gmail.draft.send
  version: "0.3.2"                     # a label for people; the digest tells code apart
implementation:
  entrypoint: agent:GmailAgent         # module:Class inside the package
  dependencies: ["requests>=2.31,<3"]  # built by the builder user, index only
network:
  hosts: [gmail.googleapis.com]        # the whole of where its code connects
resources:
  secrets:
    - id: google                       # a slot private to the install, not a credential
      binding: {cardinality: one, required: true}
      oauth: {provider: google, scopes: [openid, …/gmail.modify, …]}
  data:
    - id: draft                        # kept as agt_…__draft, per install
      fields:
        - {name: status, type: select, storage: keys, options: [prepared, sent, …]}
        - {name: body,   type: object, storage: values}   # encrypted at rest
tools:
  - id: draft
    resources: {secrets: [google], data: [draft]}      # the tool's ceiling
    functions:
      - id: send
        permission_level: 3            # leaves the platform: a card below trust 3
        timeout_seconds: 60            # the host's clock; paused while a person is asked
        resources:
          secrets: {google: use}
          data:    {draft: [read, update]}              # no create, no delete
        inputs:
          type: object
          additionalProperties: false
          required: [draft_ref]
          properties:
            draft_ref: {type: string, x-resource: {type: data, id: draft}}
        outputs: {type: object, required: [sent, status], …}
```

Figure 7.1 An excerpt of the Gmail agent's manifest from `decentai-agents`, condensed into flow style; comments added. The function's canonical name is `gmail.draft.send`: agent, tool and function ids joined by dots. That name is what the model calls, what a card shows, what a grant names and what the audit trail records.

Four ideas carry most of the contract. **Resources appear twice.** The tool lists what its functions may touch at all, and each function maps resource ids to the operations it performs (`list`, `read`, `use`, `create`, `update`, `delete`). A call's resource object is built from the function's map alone, so `send` cannot delete a draft although a sibling can. **Fields are `keys` or `values`.** Keys are plaintext, scalar and filterable. Values are encrypted at rest, may hold objects, and in a secret are never shown to a person again. **Hosts** are names, `*.name`, `name:port` for protocols other than the web, `{from_secret: slot.field}` for a host only the person knows, `[]`, or `any`. **Scopes** name a dimension a grant can narrow, such as Note's `notebook`, bound per function to a top-level input with `from_input`.

FUNCTION KEY	WHAT IT OPENS	RULE THE VALIDATOR HOLDS
`permission_level`	Nothing; above the chat's trust level, the call waits on an approval card	Required; 0, 1, 2 or 3
`timeout_seconds`	The host's clock for one call	Positive integer; default 60
`authorization.scopes`	A grant may limit the values read from an input	Binds a top-level input; required unless `required: false`
`llm`	`call.llm`, the chat's model through the platform	Not with `schedulable` or `watch`
`schedulable`	The clock may run it unattended	Level 0 or 1, no `llm`
`credentials`	`call.credential`: a login asked for as it works	Boolean
`code`	An allowed code card opens its hosts and `call.install`	Boolean
`watch`	Called without a model when the person opens the agent's screen	Level 0, no `llm`; inputs take an `action`

TABLE 7.1 What a function may declare. Each flag reaches further than a reviewer would otherwise assume, so each is opt-in and visible at review.

The permission level is a price, not a permission. The platform checks only that it is 0–3; the reviewer judges whether it is honest. At the standard chat trust of 1, levels 0 and 1 run and levels 2 and 3 raise a card.

7.3 Validation

One validator runs when a source is read, when an agent is installed, and when the runtime and each sandbox load it. It returns errors, each naming its path (`tools[0].functions[2].inputs: …`), and never raises. The rules worth naming:

- **JSON only.** An unquoted YAML date, a non-text key, NaN, or nesting past 32 levels is refused first, because approval hashes the document as JSON.
- **Ids.** `agent.id` matches `^[a-z][a-z0-9_]{1,31}$`; secret ids are at most 34 characters and data or file ids 94, so `<ref>__<id>` fits the stores. `agent`, `close`, `id` and `logger` are taken by `ToolBase`.
- **Bounds.** At most 16 tools of 25 functions; at most 50 hosts, lowercase names, never an address, scheme, path or bare `*`. No `network` block, no approval.
- **Schemas.** A closed JSON Schema subset, `object` root, depth at most 5; `$ref`, `allOf`, `anyOf`, `oneOf`, `not` and conditionals are refused, because a schema a reviewer cannot read at a glance is one nobody checks.
- **Storage.** A `secret` or `object` field in `keys` is refused; a `from_secret` host must name a string kept in `keys`, so where an agent connects stays visible.

> I-7.1 Every component judges a manifest with the same validator and the same fingerprint function, so no two parts of the platform can disagree about what an agent declared.

7.4 Sources and catalogs

An administrator saves a git repository on **Agents → Marketplace**. The Data Core fetches it at one commit, depth 1, reads `decentai-agents.yaml` at its root and validates every manifest it names. A catalog lists 1 to 100 entries, each an id and a path and nothing else: what an agent is called is its manifest's to say. The whole catalog is refused if an entry's id differs from its `agent.id`, a path leaves the repository, or anything is a symbolic link. A manifest that breaks a rule refuses only its own agent, which is listed with its reasons. **Reading a source never executes its code.** A source belongs to the organization that saved it.

7.5 Review and approval

The review shows what the manifest declares: dependencies, levels, flags, scopes, secrets, data, files, hosts, and the README up to 20,000 characters. An administrator holding `agents:agent:install` approves the agent by name, and the install works in a fixed order:

1. It fetches the commit whose catalog the page showed. If the source was read again since, the request is refused (`source_moved`, 409) and the agent must be reviewed again.
2. It validates again, and refuses the same version with a different manifest (`immutable_version`).
3. It derives secret slots private to this approval, and writes the row with `manifest_hash`: SHA-256 of the canonical JSON, keys sorted.
4. Only then does it pack the folder and store it under `(organization, digest)`. If that fails, or the same version arrives with other bytes, the row is put back as it was.
5. It pins `package_digest`, grants the installer, reclaims unpinned archives, audits `agent.installed`, and marks the row *preparing*.

No runtime is contacted; whether the code loads is the node's own report (§8.3). Archives are kept per organization so that one organization's deletion, retention or compromise cannot reach another's code. Deduplication happens on the node instead.

> **I-7.2** An approved version names exactly one manifest hash and one package digest. Nothing of an agent runs before an approval names it, and a version is never approved again with other content or other bytes.

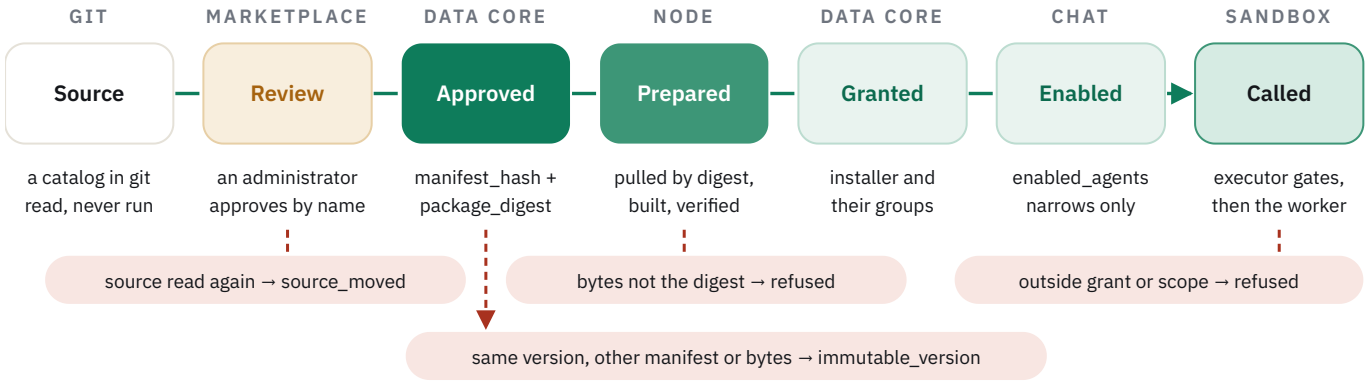


Figure 7.2 The lifecycle of an agent. The review waits on a person (amber). Approval pins two fingerprints in the Data Core (deep green). Each node pulls and verifies the bytes on its own clock (§8.3). Grants and a chat's list decide who reaches it, and only then does agent code run (pale green). Red marks the refusals at each step.

7.6 Two names: refs and qualified ids

The author's `agent.id` decides nothing on the platform. At first install the platform mints a **ref**, `agt_` and twenty random characters, derived from nothing, so nobody can predict one or compute another install's. It is found again at every update by organization, source and catalog id, so an update keeps the agent's records and grants. Everything outside the package speaks the ref: grants, the collections `agt_…__draft`, secret slots, function names, cards, the trail. Only the manifest lookup and the worker call translate it back, so the code only ever sees `gmail`. Naming an agent after another gives it nothing of that agent's.

A ref cannot be typed, so each install also gets a **qualified id** for people and policies: the bare author id when it is free in the organization, else `<source>__<id>`. It is assigned once and keys nothing, so a later install cannot relabel it.

7.7 Grants, scopes and the chat

Grants are rows keyed by the agent, not statements in IAM policies, because an agent's functions appear, change and vanish with installs, and names written into authored policies rot. An uninstall deletes the agent's grants and credential grants with it. A grant has an owner (up to 50 groups and 50 users), functions (`"*"` or up to 200 `tool.function` names) and `constraints`, the values each scope may take. Grants are allow-only: absence is denial.

On **first install only**, the install writes the smallest grant that is obviously right: the installer's groups and the installer by name. A repair leaves grants alone, so access arranged since is never undone. Wider access is edited on the agent's **Access** tab. `"*"` follows the manifest as it grows; a list keeps a new function closed until someone opens it.

At call time the executor reads each scope's value from its input, normalizes it, and holds it to the grants; a constrained allow does not hold for a call that gave no value. Credentials follow the same rule: a saved credential reaches an agent's private slot only through a revocable credential grant that a person writes.

A chat's contract joins three filters: the organization's installed agents, the grants that reach this person, and the chat's `enabled_agents`. A person's default may name only installed agents, and a chat may narrow it but never reach past it. An agent with no function left after the join is absent from the chat.

> **I-7.3** A person reaches a function only through a grant row on that agent and a chat that enables it. A chat's list can narrow what a person may use and can never widen it.

7.8 Versions and updates

A version is immutable in its manifest and its code; every change, a sample sheet included, is a new `agent.version`. An update is offered only for agents whose own manifest changed, and taking it is a new approval. Credentials saved under old shapes follow where that asks nothing new of their owners. At its next call the runtime sees the new digest and retires the old worker. A call still in flight there ends with an honest error and is not retried, because nobody can know what a half-finished call changed. A renamed `agent.id` is a different agent, which no installation follows.

7.9 What agent code is given

The SDK, `decentai_sdk`, has no dependencies, is installed into every agent environment, and is the only platform code an agent may import. `call` carries no identity of the person and no model key. A refusal raises `ResourceDenied` with the reason in words.

`call.inputs`	Validated against the manifest, with defaults applied, before the code runs.
`call.resources`	Records (`list_data` … `delete_data`), files (`read_file`, `create_file`, …) and secrets (`use_secret`), each checked against this function's operation map. Writes are checked against the declared fields.
`call.progress(text)`	A line the person sees while the function runs; nothing is kept.
`call.ask(q, choices)`	A card in the agent's name: at most 1,000 characters and 8 choices of 100. Unanswered after a day it returns `None`; the function's clock stops while it waits. With `expects="file"` the answer is a file ref.
`call.post(text)`	Speaks to the person now: at most 4,000 characters, three posts a call.
`call.llm(…)`	The chat's model, reached through the platform; the stored key never enters the worker. Requires `llm: true`.
`call.credential(host, fields)`	A login for a site the agent could not know beforehand, 1 to 20 fields, consented per agent and per site; the model sees labels only. Requires `credentials: true`.
`call.propose(code, purpose)`	Code up to 20,000 characters on a card, with the model's reading of it beside it. With `code: true`, an allowed card opens the hosts it named for that call, and `call.install` installs exactly the packages it named.
`call.screen`	A picture the person can watch and take over (§8.11).

Without a chat, as in a test, `ask` and `propose` return `None` and `post` returns `False`, so the same code runs everywhere.

7.10 The catalogue

The project's repository, `decentai-agents`, is an ordinary source; its catalog lists 38 agents, written against the same contract as a third party's.

AGENT	WHAT ITS MANIFEST SHOWS
Note	`hosts: []` ; a typed secret against a simulated remote; a `notebook` scope bound with `from_input` , optional on `find` ; all four levels, from `notes.find` (0) to `sync.push` (3).
Notebook	`user_access: [create, update]` on records; file slots with `constraints` ; a `schedulable` level-1 function with no model.
Gmail, Outlook	A connected account by `oauth` , its refresh token kept in the Data Core; one host; `draft.send` at level 3; an inbox watch that wakes a schedule only for new mail.
Mail	IMAP through the proxy with `decentai_sdk.net.Tunnel` : hosts with ports, and `{from_secret: account.imap_host, port: 993}` .
Browser	`hosts: any` ; `browse.run` at level 2 with `llm` and `credentials` ; a level-0 `watch` function for the live screen.
Code (`code_runner`)	`hosts: []` and `code: true` : whatever the program reaches is what a person allowed on its card, for that call.
Tasks, Expenses	A model that reads and code that decides: every quote checked against the source, dates only when exact; in Expenses, money in `Decimal` .

TABLE 7.2 Agents in `decentai-agents` worth reading for one pattern each.

Two show the edge of the contract. Every Google agent asks for the same OAuth scopes so one account can be granted to each; what keeps Gmail to mail is not its token, which could reach Drive, but that `gmail.googleapis.com` is the only host its worker may reach. The Code agent shows the reverse: an empty host list, widened for one call by a person's yes.

> **I-7.4** The runtime never imports agent code and never takes a worker's word for what it may do. Every resource, credential, host and model completion is granted host-side, per call, from the approved manifest's entry for the function now running.

CHAPTER 8

Execution nodes and the sandbox

The manifest decides what an agent is *given*. The sandbox bounds what its process can *take*, whatever its code tries. Agent code runs on an execution node, in a container that holds nothing else and has no route out but one proxy, as a user of its own, behind a file fence and a system-call filter.

8.1 The execution node

An execution node is one AI Runtime and its Execution Sandboxes, deployed together. The bundled node of a single-machine install is three Compose services. `ai-runtime` runs each chat's assistant, the executor and its gates, the agents' proxy (port 8002) and the sandbox registry (port 8005). `sandbox` is the same image started in another role: it runs the spawner (port 8003) and every worker. `node-issuer` holds the node-authority private key with no network at all and writes the node a fresh credential into a volume the runtime mounts read-only. A node on its own machine is the same three services, dialling a Data Core elsewhere, with no port open.

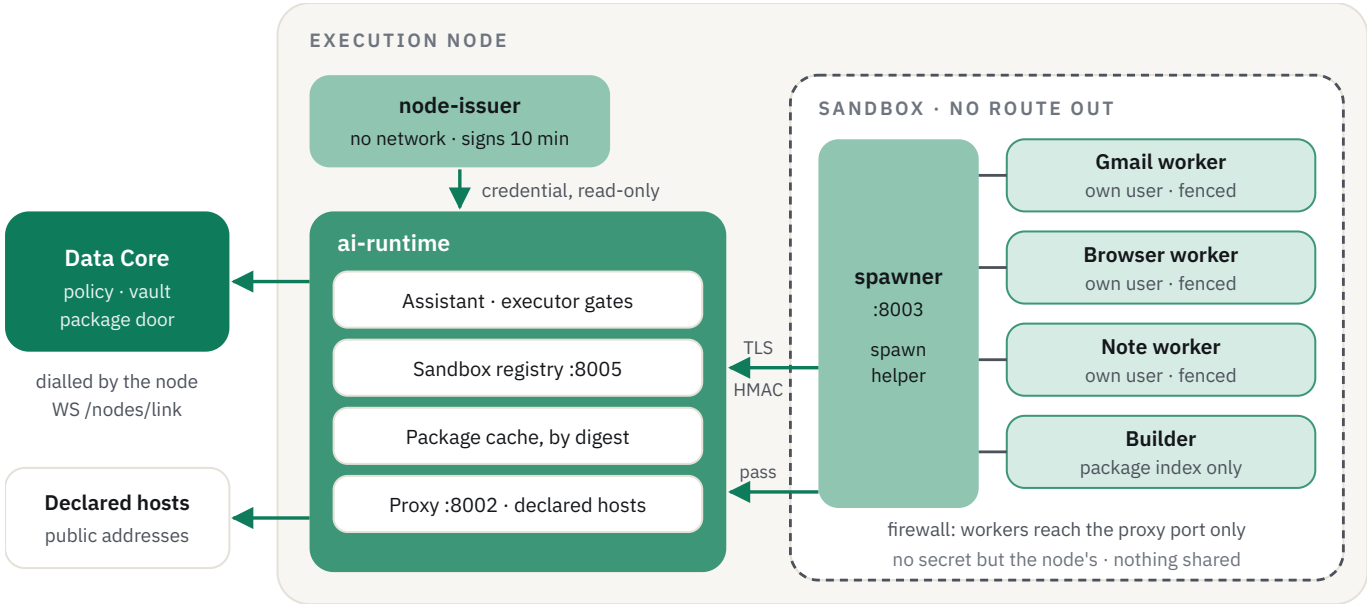


Figure 8.1 An execution node. Every connection is dialled outward. The runtime dials the Data Core's node link, and the sandbox dials the runtime's registry over TLS 1.3 with HMAC proofs of the node's secret. Workers' connections leave the sandbox only as passes to the runtime's proxy, which reaches declared hosts at public addresses. The dashed outline is the trust boundary: the sandbox sits on an internal network with no route to the internet, the database or the Data Core.

8.2 How a node links to the Data Core

Nobody registers a node. Trust is given once to an **authority**: a public key or a JWKS issuer, scoped `shared` or to one organization. Any node presenting a credential it signed is admitted, so a thousand nodes need no step each. The credential names the node, audience `decentai-data-core`, and optionally an organization and labels. It lives at most an hour; the bundled issuer signs ten minutes. The node never holds the authority's key, so a node taken over can sign for nobody and keeps its name for minutes.

The node keeps one WebSocket to `/nodes/link`, reconnecting after 1, 2, 4, 8, 15, then 30 seconds. The Data Core holds no node's address, so a node can sit behind a firewall that admits nothing. Everything the Data Core sends is signed: a reply names the node, its request id and the digest of the reply's bytes; a link frame names the node, the connection's nonce and its sequence. A node treats anything else as silence, because a reply can be a contract. A heartbeat every 20 seconds renews the node's chat leases, which last 60; every call for a chat carries its lease number, so two nodes can never act for one chat.

Code arrives through the **package door**, the only verbs a node principal may use: `Agents:Package:List`, the digests standing approvals pin; `Agents:Package:Fetch`, bytes by digest; and `Agents:Package:Prepared`, ready or failed. The runtime lists every ten seconds, which turns *Preparing* into *Ready* with no chat involved.

> **I-8.1** Nodes dial out and are admitted by authority, never by address. A node acts on nothing the Data Core's key did not sign for that node, and on no chat whose live lease it does not hold.

8.3 Environments built from approved digests

The runtime caches packages by content, one folder per digest. It checks bytes against the digest before writing, and the unpacked manifest against the approved hash before sending anything on. Tampered bytes therefore fail for everyone instead of poisoning a copy that many organizations share, and the runtime runs nothing from the cache.

Each sandbox keeps its own copy, and no folder is shared. Asked for a digest it lacks, it asks its runtime for the bytes. It checks digest and hash before writing, unpacks, and builds an environment keyed by the dependency list and interpreter, so a code-only update builds nothing. Then it starts one worker as installation's user, the **verification start**, to prove the package loads. A failure removes what its step made. Dependencies take two users, because a build is code too. The **builder** downloads and builds wheels through a pass that opens only `pypi.org` and `files.pythonhosted.org`. The spawner unpacks them and runs nothing of theirs.

> **I-8.2** Code reaches a sandbox only through its runtime, and is checked against the approved digest and manifest hash, both on the runtime and in the sandbox, before anything is written.

8.4 The worker pool

The worker pool keeps one worker per **approval**, not per digest, so nothing an agent holds in memory crosses organizations. A worker starts on first call, is kept warm for every chat of its organization, and keeps per-chat state such as a browser under `call.conversation`. It is disposable: replaced when its code changes, it crashes or it is ended for memory. The runtime places a new worker among *eligible* sandboxes: healthy, locks proven in the current process, not draining, holding the labels asked for. It prefers the one already running the agent, then one holding its package, then the least loaded (memory in 256 MB buckets, then processors in tenths), then takes turns.

8.5 The worker protocol

The worker is the SDK's worker loop, run by the agent environment's interpreter in isolated mode. Its argv is empty and its environment an allow-list that never holds the model key, the Data Core's trust key or the runtime's settings. Host and worker exchange newline-delimited JSON, protocol 2, at most 2 MiB a line; a malformed line ends the worker. Every worker request names its `call_id` and is answered from that call's operation map, only if the call was sent to this worker. Bytes whose base64 exceeds 256 KiB travel ahead as `bytes` pieces.

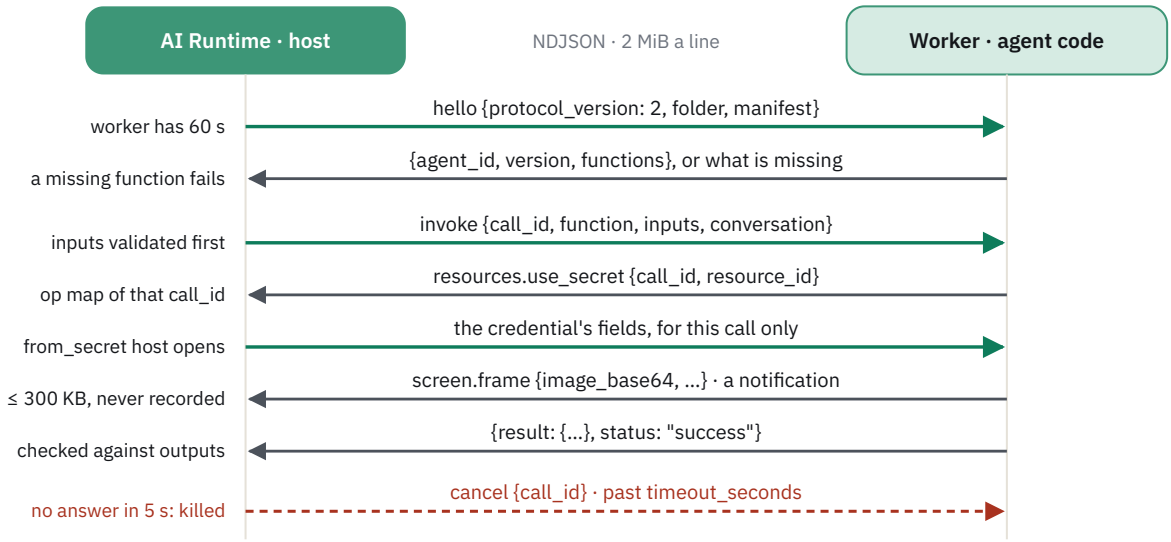


Figure 8.2 One call over the worker protocol. Green lines are the host's, grey the worker's; the spawner carries every line unread. The notes on the left are what the host checks or does at each message. Killing a worker fails every call in flight on it, and none is retried.

8.6 The runtime–sandbox link

Only its runtime commands a spawner. The two share one node secret, `AI_RUNTIME_SANDBOX_SECRET`, which Compose gives to nothing else, the Data Core included. The secret never travels. The side spoken to first sends a challenge, and the other answers with an HMAC-SHA256 under the secret with its role written in front, so one side's proof never passes as the other's. Every connection is TLS 1.3 with a key and certificate each process makes at start, so no certificates are issued or signed. Each proof covers the fingerprint of the certificate seen on that connection, so anyone in between fails both ways. A sandbox proves itself first on every request, and removes the secret from its environment so no worker inherits it.

8.7 Confining a worker

A small C program, the spawn helper, does the one thing the platform's user cannot: start a child as another user. It is compiled because a binary can carry that right and a script cannot. It refuses callers other than the platform's user and uids outside 20000–29999. In order, it sets limits, sets no-new-privileges for everything it will start, becomes the agent's user, applies Landlock to the paths the spawner named, and applies seccomp last. Each approval has its own uid per sandbox; the first uid is installation's, the last the builder's. A home is emptied, as its user, before every start.

Landlock lets a worker read the system, the browsers, its package and environment, and write its home. From ABI 4 it may connect over TCP only to the proxy port; from ABI 6 it reaches no abstract socket it did not create. Seccomp is a deny-list, because an allow-list would be a list of what somebody forgot. It refuses namespaces and mounts, `ptrace`, `bpf`, `io_uring`, module and keyring calls, and unneeded socket families. Each sandbox proves this at start and registers with what it proved. The runtime places no worker where confinement failed, and an agent's page states what a weaker kernel cannot enforce.

8.8 Network confinement

Three things leave the proxy as a worker's only way out. The sandbox's network is internal and joins only the runtime. At container start, the image's first step runs as root, the one step holding `NET_ADMIN`, and writes an nftables chain: for socket owners in uids 20000–29999, TCP to `127.0.0.1` on the proxy port is accepted and everything else rejected, DNS and the spawner included. It then becomes the platform's ordinary user and removes the capability from its bounding set, so nothing started afterwards can regain it. The spawner relays that port to the runtime's proxy, unread.

The proxy knows a worker by its pass and refuses a host the manifest does not name. It resolves the name itself, refuses the host if any answer is internal, and connects to the address it checked; it never opens TLS. Two kinds of host are lent per call: a `from_secret` host when `use_secret` hands over the credential, and a code card's hosts on the person's yes. Both close when the call ends. Every connection is written down with its owner, destination, bytes and duration.

> **I-8.3** A confined worker's only route off its machine is the runtime's proxy, and the proxy admits only hosts the approved manifest, a credential handed to the running call, or a card the person allowed names, at public addresses.

8.9 Limits and timeouts

No manifest asks for these numbers. CPU time is not limited, because the limit counts over a warm worker's whole life; the host's clock bounds each call instead.

BOUND	VALUE	ENFORCED BY
Processes, per agent user	2,048	the spawn helper, per user
Open files / size of one file / core files	4,096 / 1 GiB / 0	the spawn helper, per process
Memory and processors, all agents together	`SANDBOX_MEMORY`, `SANDBOX_CPUS`	the container engine (0 is none)
Largest agent ended at	90% of the sandbox's memory, looked at each 1 s	the spawner
Disk an agent keeps	measured every 30 s, not limited	the spawner, as the agent's user
One line of the worker protocol	2 MiB; pieces 256 KiB; 16 streams, 256 MiB	whoever reads the line
Proxy connections	128 per worker, 1,024 in all, 256 unidentified	the proxy
Proxy: naming the destination / upstream connect / idle	30 s and 64 KiB / 20 s / 30 min	the proxy
Package	5,000 files, 64 MB uncompressed	the Data Core, when it packs
Screen frame / input events per message	300 KB / 64	the SDK and the host

TABLE 8.1 What a worker is held to.

CLOCK	VALUE	THEN
A function call	`timeout_seconds`, default 60 s	`cancel`; killed after a 5 s grace
Handshake / verification start	60 s / 120 s	worker killed / install fails with the reason
Dependency build	300 s (`AI_SANDBOX_BUILD_SECONDS`)	ended with everything it started
Sandbox heartbeat	2 s; late after 10 s; gone after 30 s	given nothing new; agents placed elsewhere
Reaching a spawner / first sandbox at start	10 s / 120 s	next sandbox tried / runtime does not start
Failed confinement check	rechecked every 30 s	no worker until it passes
Drain: stopped sandbox / administrative	60 s / up to 600 s per worker	what is left is ended
Node heartbeat / lease / stocking	20 s / 60 s / 10 s	three missed beats and the chat moves
Node credential	≤ 3,600 s; bundled 600 s	link closed at the next heartbeat

TABLE 8.2 The clocks, all owned by the host and never by the worker.

> **I-8.4** The host owns every clock. A worker cannot extend its own time, and a worker that does not answer a cancel is killed with everything its user runs.

8.10 Draining, many sandboxes, many nodes

One runtime serves many sandboxes. A second on the same machine is one more Compose service; one elsewhere needs only the node secret and the runtime's address. `Nodes:Node:Drain` drains one sandbox or all: no new worker starts there, running workers finish and retire, and the agent's next call is placed elsewhere. Sandbox labels such as `gpu` filter placement, though no manifest can ask for one yet. Labels on a node's credential are shown and gate nothing, because what a node says of itself is reported, not believed.

The Data Core places a chat on the node holding its lease, else among connected nodes that may serve the organization: its own first, then the chat's last node, then the least loaded. With `placement: own_only` nothing of the organization reaches a shared node. A chat that moves is rebuilt from the record; a call running on the old node is reported unknown, never rerun.

8.11 The live screen

An agent that drives something visible shows it with `call.screen`, undeclared, since a picture shows nothing a file could not. A frame, a JPEG or PNG of at most 300 KB, goes as `screen.frame` to the host, as `screen_frame` up the node link, and to the page. It is relayed, never recorded, and dropped rather than queued. The Browser agent sends 1,280-pixel frames: one per step, one every 0.35 s while the person holds it, one every 100 ms while they drive. Input returns the same road. The page sends `AI:Chat:Screen`, the Data Core checks that the person may write in the chat, and the node hands the events to the call that showed the screen, dropping them once it ends. **Take over** pauses the run; **Hand back** resumes it with the agent told to look again. Opening the screen calls the chat's `watch: true` function without the model, hence level 0 and no `llm`. Nothing of the browser is reachable from the person's machine.

WHAT THE SANDBOX DOES NOT DO

It does not separate the people of one organization: a worker serves every chat of its approval, and a host lent to one call is reachable from another running beside it. It does not stop an agent misusing what it was rightly given. Memory is shared among agents and enforced by ending the largest. Before kernel 6.12, two agents that both intend to can share an abstract socket. The container, sharing the host's kernel, remains the outer wall.

CHAPTER 9

Secrets and data

Everything a person owns lives in one database, under one implementation of "a document somebody owns that somebody else may be allowed to see". Secrets go further: their values reach an agent's code for one call, and never a person or the model.

9.1 The data layer

The Data Core keeps its state in one MongoDB database of **45 collections**. Each collection is declared, with its validator and indexes, in the Data Core's database schema. At start the Data Core creates what is missing, applies each validator (`validationAction: error`) and drops any index the schema does not declare. A database is therefore exactly what the code says it is.

PURPOSE	COLLECTIONS	N
Identity and access	`organizations` `users` `groups` `roles` `policies` `sessions` `api_keys` `invitations` `password_resets` `signups` `auth_throttle` `first_run`	12
Chats and the assistant	`ai_chats` `ai_messages` `ai_chat_events` `ai_chat_storage` `ai_approvals` `ai_schedules` `ai_memories` `runtime_sessions`	8
Agents	`ai_agent_sources` `agent_manifests` `agent_grants` `agent_secret_grants` `agent_samples`	5
The data layer	`secrets` `secret_definitions` `agents_data` (records) `files` `skills` `mcp_servers`	6
Connections	`llm_connections` `oauth_apps` `oauth_states`	3
Execution nodes	`node_authorities` `execution_nodes` `refused_nodes` `chat_leases`	4
Reach	`webhooks` `webhook_deliveries` `webhook_subscriptions` `webhook_sends` `push_subscriptions`	5
Evidence	`ai_audit` `llm_usage`	2

Thirty-three collections carry `org_id` in their schema. The remaining twelve either hang off a row that does, such as a chat's stored results or a webhook's deliveries, or belong to the deployment rather than to a tenant: nodes, the first-run lock and sign-in throttling.

The six data-layer domains share one document shape. `resource_ref` names one document. `resource_id` is a category label that many documents share. `owner` is a map of groups and users. `keys` holds plaintext metadata that can be queried, and `values` holds the encrypted payload. A field in `keys` can be searched and shown; a field in `values` cannot be searched at all. A document is bounded to 50 entries, 4 MiB per value and 10 MiB for the whole encrypted map. That stays inside MongoDB's 16 MB limit.

Visibility is a filter, not a check. Every read built for a caller composes its query from the organization, the domain and the owner map. The query therefore cannot match a document the caller may not see. A check after a lookup can be forgotten; a filter inside the lookup has to be removed on purpose. The *Everyone* sentinel resolves against the reader's own organization, so an organization-wide share never crosses into another tenant.

Two rules govern writes. Nobody may share beyond their own reach, meaning their groups and the peers who share one of them. Only the creator, the *steward*, may edit, re-share or delete a document. A per-domain escape grant lifts both rules, and stewardship can be transferred. One sharing engine applies all of this through two profiles. `PERSONAL` covers secrets, records, files and skills. `INFRASTRUCTURE` covers model connections and agent sources.

9.2 Encryption at rest

Every encrypted field is sealed with AES-256-GCM, using a 32-byte key and a fresh 12-byte nonce. The envelope stores `alg` , `key_version` , `nonce` and `ciphertext` . The document's `_id` serves as associated data, so a ciphertext copied into another row fails its tag instead of opening there. The binding is to the id rather than the owner, because owners are edited.

Keys come from `SECRET_ENCRYPTION_KEYS`, a comma-separated list of `version:base64-key` pairs. `SECRET_ENCRYPTION_ACTIVE` names the version used for new writes and defaults to the highest. Versions compare as text, so a map with mixed widths (`1`, `2`, `10`) is refused, because it would make a stale key active. If the variable is unset, the Data Core does not boot. A key derived from another setting would seal every credential under a secret chosen for something else.

Rotation takes three steps. Add a version with the key generator's rotate option and restart, so new writes use it. Rewrite everything onto the new key with the re-encryption tool, after a `--check` pass that counts first. Only then retire the old key. The tool asks every store for its own list of encrypted fields: data-layer values, model-connection keys, OAuth client secrets, a source's git token, and webhook and subscription secrets. A hand-kept list once drifted, and a field such a list misses becomes silent, permanent loss when its key goes. A blob whose key is missing is marked unreadable for *display*, so one bad row does not blank a page. For *use*, the failure is loud: a 409 that says how to fix it. An agent acting on an empty credential is worse than one that stops.

> **I-9.1** A sealed value opens only in the document it was written for, under a key version the deployment still holds. Nothing falls back to a default key or returns partial plaintext.

9.3 Credentials

A secret is created against a **definition** (`secret_definitions`), which lists the secret's fields and says which are encrypted and which are required. Definitions are derived from manifests, never written by hand. Each lives in a private namespace (`agt_<ref>__<slot>`), so a category names exactly one approval. An earlier design allowed shared "family" slugs, and any repository could copy two field names and read what was stored under them. Definition versions are immutable. After an agent update, a credential follows the new version only if the new shape asks nothing new of its owner. Otherwise it is marked *outdated* and refused at use, with a sentence saying what is missing.

A credential's steward is its creator. Sharing it lets colleagues' chats *use* it, not edit it. A **grant** (`agent_secret_grants`) points one saved secret at one agent's declared slot. It is offered only where the two shapes match exactly. Several agents can then read one key with no second copy, and a rotation stays one act. At use, the Data Core picks the instance in a fixed order: the chat's binding, a grant to that agent, the person's default, then the only visible instance. When several remain, the answer is an ambiguity error, never a guess.

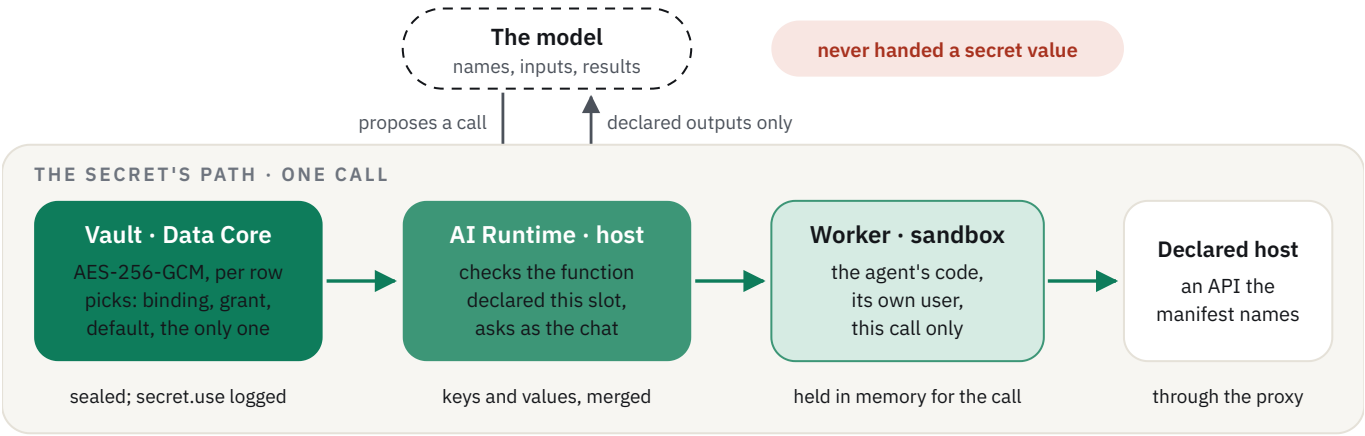


Figure 9.1 The path of a secret. `Secrets:Secret:Use` answers only a runtime principal; any other caller gets 403. The runtime passes the values to the worker as the answer to the worker protocol's `resources.use_secret`, which only a function that declared the slot may send. The model is outside the path. It proposes a function by name with inputs, and sees only the outputs the manifest declared, after validation.

A worker starts with no credentials. When a function asks for its slot, the host's resource broker checks that the function declared it, then calls the Data Core as the chat's delegation. The Data Core decrypts in-process, writes `secret.use` with the ref to the audit trail, and returns keys and values together, because a password without its hostname is half a credential. The values reach the worker's memory and nothing else. They do not appear in the transcript, the activity stream or the trail. The credential's host is opened at the proxy for that one call.

One limit remains: a warm worker could keep a value between calls. An access token's expiry bounds that risk, but nothing bounds it for a typed key. This is why agents are read before they are approved.

> **I-9.2** A secret's values leave the Data Core only through two runtime-only operations, `Secrets:Secret:Use` and `Secrets:Credential:Resolve`, and only toward the worker of a function that declared the slot. No read by a person returns them, and the model is never on the path.

9.4 Connected apps

Some credentials are granted rather than typed. A definition with an `oauth` block names the provider, its addresses and its scopes. The platform adds the fields it fills itself: `account`, `access_token`, `refresh_token`, `expires_at` and `status`. The client **registration** belongs to the organization (`oauth_apps`, one per provider id, with the client secret encrypted and write-only). The registration also pins the provider's addresses. An agent that names other addresses for the same provider cannot connect, because the client secret and the person's tokens would go to that agent's server. The person's **grant** is the credential.

The flow belongs to the platform, not the agent. `Secrets:Oauth:Start` writes a single-use state row (`oauth_states`, valid ten minutes) holding a PKCE verifier and the browser session that asked. The provider returns to `/oauth/callback`, which spends the state, requires the same person in the same session, exchanges the code, and creates the credential or refreshes it in place.

The web client lives on its own origin, and its session token cannot ride in a popup, so it starts with `client: web`. The callback then parks the code behind a second one-use hand-off that lasts two minutes. The popup hands it to its opener with `postMessage`, addressed to the `PUBLIC_APP_URL` origin alone. If that setting is unset, the opener learns only that the flow failed. The web client spends the hand-off with `Secrets:Oauth:Complete` under its own session, and the same-person check runs there. The phone app follows the same path with `client: native`.

At use, the Data Core first refreshes an expiring token, and the agent receives the account and an access token. The refresh token never leaves the Data Core. A revoked grant marks the credential `needs_reconnect`.

9.5 Records and files

Records (`agents_data`) are what an agent keeps for a person, such as a watcher's memory of what it has seen. Each follows a shape declared in the manifest, and a person's write is checked against that shape just as the agent's own writes are. A person may write a record only where the manifest's `user_access` allows; otherwise they can read and delete. An agent reaches records and files only in its declared slots. A document under any other category answers as nonexistent. The exception is a ref the call was *handed*, in its inputs or in the person's answer, which the agent may read but not write or delete. The web client exports a kind as CSV or JSON and leaves out fields the manifest marks secret.

Files own bytes as well as a document, and upload is the only way to create one. A storage location taken from a caller could point anywhere. The storage id is derived from the bytes, the name, the purpose and a folder that embeds the uploader. The bytes are deleted when the last document referring to them goes. Uploads are capped by `MAX_UPLOAD_MB` (default 25). A chat attachment is an ordinary file, passed to a function by ref.

`FILE_STORAGE_PROVIDER` selects `local` disk, `s3` (including any S3-compatible store, optionally with KMS), `azure_blob` or `gcs`. Each document records the connector that wrote it, so a store the deployment moved away from is still read. Containment is enforced by the connector itself: the disk refuses any path outside its root, and a bucket refuses a folder that climbs (`..`). Memories, chats and execution nodes sit outside this layer on purpose, so its sharing model cannot reach them by accident.

9.6 What leaves a deployment

Nothing goes to DecentAI's makers. Data leaves on the following routes, and a named role decides each one:

- A chat's messages go to its model provider.
- A chat's messages, its model key and its granted credentials go to the execution node that holds it. *Own nodes only* keeps an organization off shared nodes.
- An agent's traffic goes to the hosts its manifest declares, through the proxy.

- Sign-ins go to OAuth providers, and a person's MCP servers are reached at their own addresses.
- Notifications pass through push services and the mail server.
- Outbound webhooks go to the addresses a person subscribed.
- Speech goes to a provider only where an administrator chose one.

The vault, the trail, files, records and chats stay in the deployment's own storage.

CHAPTER 10

Evidence, monitoring and audit

The model's words are its own. Whatever the platform asserts about a turn comes from records it wrote: what the chat saw happen, what was allowed and done, what the processes did underneath, and what the models used.

10.1 The activity stream of a turn

A chat keeps two logs in `ai_chat_events`. The **inbox** (`direction: in`) holds every event the assistant must absorb: a message, a finished job, a fire, a webhook. Each event is written with a per-chat sequence number before the assistant hears it, so a session rebuilt after a crash resumes from its bookmark. The **outbox** (`direction: out`) is the audience's replayable log. Its frames are defined once in the shared chat contract and include `message_created`, `working`, `activity`, `plan_updated`, `approval_requested`, `sleeping` and `stopped`.

An `activity` line has a kind: `call_started`, `call_finished` (with status and duration), `agent_progress`, `job_*` or `helper_*`. Lines of one call share its `call_id`. The Data Core validates each event as it records it, and a malformed event costs the replay, not the live frame. Screen frames are never recorded.

10.2 The audit trail

`ai_audit` is append-only and written at the Data Core's chokepoints. A line records `event_type`, organization, person, actor, chat, execution id, function, `resource_refs`, `details` and time. The Data Core writes its own types: `secret.use`, `llm.use`, `approval.requested` and `.resolved`, installs and grants, `chat.stopped`, `schedule.*`, `webhook.*` and `work.stopped`. Every other gateway action that changes state, refused or not, becomes `platform.action` with its outcome (`success`, `denied` or `failed`) and a reason of up to 300 characters.

The runtime may write exactly one type, `execution`. That line records the agent, function, levels, an outline of the inputs, the status, any error and the duration, in at most 4,000 characters. It also lists the hosts the proxy counted: the first twenty, then a count of the rest. Because the runtime has only that one type, it cannot forge an approval or a credential read.

The trail is a witness, not a gate. An append is tried twice. A line the database refuses goes whole to the error log at `CRITICAL`, and the act it describes still happens.

A person reads their own trail (`ai:audit:list`), and one chat's trail appears in its *Activity and audit* dialog. The organization's whole trail needs `ai:audit:list_all`. Lines outlive their actors. No secret value is written to the trail; an agent's error text and the inputs as the model proposed them are kept as they came.

> **I-10.1** Nothing through the gateway edits or deletes an audit line. The runtime adds only `execution` lines; every other type is the Data Core's record of its own act.

10.3 The evidence behind a claim

Each plan item (at most 12) carries `evidence` and `verified`. While an item is active, the storage ref of every successful call and the id of every finished job attach to it. The model may cite only refs the trace holds, and anything else is refused. An item marked done with no evidence is shown as *unverified* rather than refused. A table or chart

appears only when the model names the stored result of a successful call and the field is a declared list with rows. The page renders those rows from `ai_chat_storage`, so no value is retyped. A successful write rides the message as a `success` part. A claim with no trace behind it remains the model's claim.

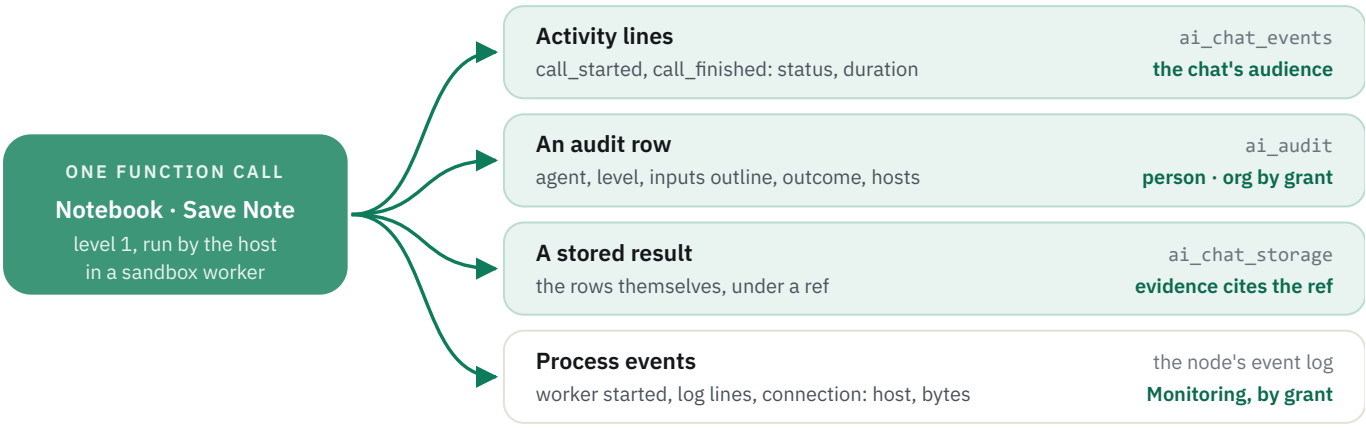


Figure 10.1 What one call leaves behind. The tinted rows are Data Core records. The white row is the execution node's own log, which the Data Core reads over the node link. A call that asks the chat's model also adds an `llm_usage` row. No row holds a secret value or the contents of an encrypted connection.

10.4 Monitoring

A second record shows what the agents' processes did. The runtime and each Execution Sandbox write JSON lines into their own folders, because no container may read another's. The kinds of line include `worker.started`, `worker.ended`, `log`, `process`, `memory` and `connection`. A connection line records the host, the resolved address, whether the connection was allowed and reached, the bytes each way and the duration. It never records contents, because the proxy does not open TLS. Each writer keeps eight files of 8 MiB, so at most 64 MiB, and a line that cannot be written never fails what it records. The Data Core reaches no node itself. It asks each connected node a `monitor` question over that node's link, and keeps each organization to its own agents' lines.

VIEW	GRANT	WHAT IT SHOWS
Overview	`agents:monitor:summary`	what the agents did in a window, summed from the audit trail, and its model calls
Tokens	`agents:monitor:tokens`	tokens, cache share, estimated cost, by model, purpose, agent and person
Agents	`agents:monitor:usage`	limits; each running agent's memory, processor, processes and disk, every 5 s
Events · Connections	`agents:monitor:usage`	every line, filtered by outcome, kind, agent, host, text and date; connections alone with bytes

No member holds these grants by default. What an agent did is what every person did through it, so seeing it is a decision someone makes.

10.5 Token accounting

Every model answer a chat receives becomes one `llm_usage` row, kept for 396 days. The row records the purpose, the connection, provider and model, the tokens and the duration. The purpose is one of `assistant`, `agent`, `title`, `summary`, `code_review`, `rerank` or `embed`. The organization, person and chat come from the delegation, never from what the runtime says.

Providers count cache tokens differently: OpenAI and Gemini include them in the prompt count, and Anthropic and Bedrock do not. The runtime therefore normalises every answer into `input` (uncached), `cache_read`, `cache_write`, `output` (thinking included) and `reasoning`. The cache share, `cache_read / (input + cache_read + cache_write)`, measures how well prompt caching works.

The session sends rows in batches of at most fifty, and no beat waits on the send. The Data Core accepts a batch whole or not at all. Cost is an estimate at the catalog's current prices. Tokens of an unpriced model are reported as `unpriced_share` and are never shown as a cost.

10.6 Package checks and health

A sandbox installing a package checks the bytes against their digest, and the manifest inside against the approved hash, before it writes anything. It then builds the dependencies with its own builder. Last, it starts one worker whose greeting proves the package loads: the **verification start**. A package that fails is remembered until the approvals change, and the Agents page shows *Preparing* → *Ready* or *Failed*.

Health is answered at three doors. The Data Core's `/healthz` checks liveness only. Its `/readyz` pings MongoDB and answers 503 without naming the database, because the route answers anyone. The runtime's only HTTP door is `/internal/health/live` on port 8001.

CHAPTER 11

Time and reach

A chat keeps working while nobody watches. A clock wakes it, outside systems may knock, and when something needs a person the platform finds them: by push, by phone, by email or at an address of theirs. Every path has a limit, and one switch stops them all.

11.1 Schedules

A schedule is a row in `ai_schedules`, and it runs in one of two modes, with a third kind of wake reserved for the assistant:

- A **wake** delivers a note, for example "morning brief", and lets the assistant decide what the moment means.
- An **invoke** runs one function the manifest marks `schedulable`, with fixed inputs and no model. Such functions are capped at levels 0 and 1, with no approvals. The run wakes the assistant only when the result's `wake_field` is non-empty, so an empty five-minute sweep costs no model call.
- A **sleep** is the assistant's own one-off wake. A chat may have one, for at most a day, and it never appears on the Schedules page. A longer wait must be a schedule the person can see and stop.

A time is given as `at` (ISO 8601, read in the person's zone), `delay_seconds`, `every_seconds` (at least 60) or a five-field `cron`. `@daily`, `L` and wrapping ranges are refused with a reason. A chat holds at most 50 rows, and a twin of an existing row is refused with that row's id.

The clock is the Data Core's. A row must fire once however many nodes hold a copy of it, and fire even when no node holds its chat. Every five seconds the clock places each due chat on a node. It then *claims* the row with one conditional write, which holds only if the row still shows the due time the clock found. Finally it sends a signed `fire` over the node link. Four timing rules follow:

- A period counts from when the row was due, not from when it ran, so an hourly row stays on the hour.
- An overdue row fires *once* and rejoins its count. A Data Core that was down replays no backlog.
- The next run comes at least 60 seconds after the fire it follows. Otherwise a miscomputed row would fire on every tick, and each fire may be a model turn.
- A row never runs twice at once. One fire may wait, and later fires fold into it as `merged`. A pause drops the waiting fire as `skipped`.

A row remembers its last 20 runs. The clock, the node and the person's own doors each write one row at a time, so none undoes another.

> **I-11.1** The Data Core fires each due row once, under a conditional claim. No node decides when, and no outage becomes a burst of missed runs.

11.2 Webhooks in

A webhook (`whk_…` , at most 25 per person) has a name, the person's instruction of up to 4,000 characters, up to 40 typed fields and one **standing chat**. Every event wakes that same chat, which keeps its plan and schedules between events. The secret (`dws_…`) is shown once. It is *sealed, not hashed*, because checking an HMAC needs the secret itself.

```
POST /hooks/<webhook id>
Authorization: Bearer <the webhook's secret>
X-DecentAI-Timestamp: 1760518000                # with a signature
X-DecentAI-Signature: sha256=<HMAC-SHA256 of "<timestamp>.<body>">
Idempotency-Key: <the caller's id for this event>   # optional

{"customer": "Acme", "amount": 1200, "invoice_id": "INV-1043"}
```

#	CHECK, IN ORDER	REFUSED WITH
1	webhook on; its person active and not stopped; organization allows webhooks	`404` one answer for all
2	body at most 64 KB	`413`
3	bearer secret, in constant time; signature where required or sent; timestamp within 5 min	`401`
4	at most 60 events a minute, counted per Data Core process	`429`
5	a JSON object, declared fields only, each of its type	`400` naming the field
6	idempotency key (at most 200 characters) seen within a day	`202` first id, `repeated`

An accepted event is answered **202** at once, so a caller never waits on a model. Each call past the first check goes into `webhook_deliveries` (kept 30 days) and onto the trail. The trail records field names, never values. The Data Core then sends a `webhook_event` frame to the node holding the chat. The instruction in it is read from the webhook's own record. The assistant receives two lines: the person's instruction, as theirs, and the posted fields as an `EVENT` marked *data, never instructions*.

> **I-11.2** What an outside system posts reaches the assistant as data beside the person's own instruction, never as an instruction.

11.3 Webhooks out

A person may subscribe up to ten `https` addresses to four events: `approval.waiting` , `reply.ready` , `run.finished` and `run.failed` . Card and reply events are queued only while the chat is closed; run events are always queued. A body carries the event, the chat's id and title, a line, a link and the time. It carries the reply's text only on request, and never a secret or the conversation.

JSON bodies are signed as inbound calls are, with the subscription's sealed `dss_…` secret. Slack and Teams bodies go unsigned, because those receivers check no signature. The outbound sender runs one thread per process and works the queue every 15 seconds. Each attempt has a 10-second timeout and follows no redirect. A timeout, a 5xx, a 408 or a 429 is retried after 1 minute, 5 minutes, 30 minutes and 2 hours, then every 6 hours, for one day. Twenty failures in a row pause the subscription.

An address is checked when it is subscribed and again before every send. The address check resolves the host and refuses it if any answer is off the public internet: private, loopback, link-local, `100.64.0.0/10` , reserved, multicast, or an IPv4 address carried inside IPv6. Anyone may subscribe, so a notification must not become a route to the database or to a cloud's metadata address. The connection itself is made by name, so DNS rebinding in the gap between the check and the connection remains a known limit.

11.4 Notifications

When a card opens or the assistant answers, and no browser of the person has the chat open, the notifier tells them. It works on a thread of its own, so no turn waits on a push service. It pushes to every device the person subscribed. Browsers receive web push signed with the deployment's VAPID pair (`VAPID_PUBLIC_KEY` , `VAPID_PRIVATE_KEY` , `VAPID_SUBJECT`) with a 24-hour TTL. Phones receive Expo push through the token they registered. A device the service reports gone is dropped.

Only when no push reached anyone does the notifier send **email**, and then at most once per chat every ten minutes. Answer pushes are spaced at least two minutes apart. Cards are not spaced, because a card means someone is waiting.

A card names the agent and the step by the approved manifest's function name (*Notebook is waiting for your approval* / *Save Note*), never by the call's address. The text passes through services outside the deployment. A person may therefore turn *Say what happened* off, and an organization may require that of everyone (`notification_text: hidden`). The notification then says only that something is waiting, with the link.

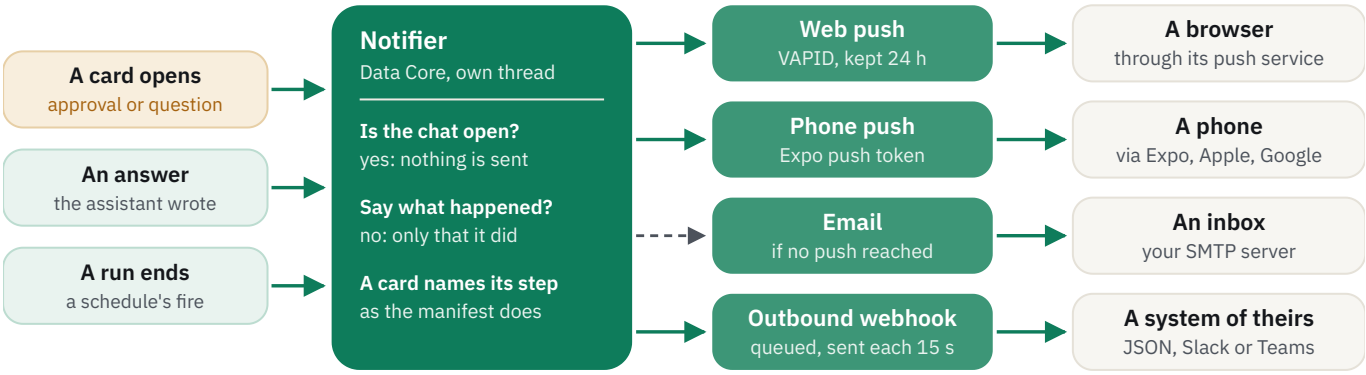


Figure 11.1 The paths a moment takes to reach a person. Pushes go to every subscribed device. Email (dashed) is sent only when no push reached anyone. Outbound subscriptions receive card and answer events while the chat is closed, and every schedule run. Each push endpoint and outbound address passes the address check before anything is sent.

> **I-11.3** No push and no outbound delivery goes to an address that resolves off the public internet. The check runs when the address is registered and again on every send.

11.5 Stop everything

The stop at the top of every page (`AI:Switch`) does not depend on anything agreeing to end. It acts in a fixed order:

1. The person is marked stopped. From then on, nothing of theirs is placed on a node and no key is minted, whether a message, a card, a schedule or a webhook asks; their webhooks answer 404.
2. Every chat a node may be serving for them is killed with `stop` and `force` . Each gets ten seconds to record what it did.
3. Every delegation of theirs is deleted. A process that ignored its end can then read no credential, key, record or file.

The stop is recorded as `work.stopped` . Resuming is the person's own act (`work.resumed`) and attaches nothing. A schedule that fell due during the stop fires once. A runtime's delegation has no control over this switch, so stopped work cannot restart itself. Stopping a single chat is narrower: it ends what is running but holds nothing, and that chat's schedules keep firing.

> **I-11.4** While a person is stopped, no chat of theirs is placed on a node and no delegation is minted for them. Work already running loses its keys.

CHAPTER 12

Clients

The engine is headless: it publishes an API and no page. Three clients speak to it — a web client, a phone app and a desktop app — over the same routes, with the same kind of session token, through the same gateway. None of them holds authority. Each shows what the Data Core says the signed-in person may see, and asks the Data Core for everything else.

12.1 One API

The Data Core's one published port answers a small, fixed set of routes. `POST /app` carries every action, named `Domain:Controller:action` and checked against the caller's policies (chapter 4). `WS /chats/{chat_id}` carries one live chat. `/auth/*` signs people in and runs the first-run page; `/upload` and `/download/*` move file bytes; `/oauth/*` takes connected accounts' redirects; `/hooks/*` takes inbound webhooks. A client is a renderer of these routes. It has no route of its own and no permission of its own.

The reason is the threat model. A client runs on a device the platform does not control, and a modified copy can send any request it likes. So nothing a client decides is trusted: reads are filtered by visibility inside the database query, and every action is resolved again from the session row on the server. A client that hides a button hides it for convenience, not for safety.

Signing in names the client. `client: "web"` (the web client) and `client: "native"` (the phone app) are handed the session token in the response body, and present it as `Authorization: Bearer`. A browser on the engine's own origin that names neither is given a cookie instead. The session row records which client holds it, so that what is kept for one client — making API keys is the web client's — can be told apart from the other. A session lasts seven days, or ninety with *keep me signed in*.

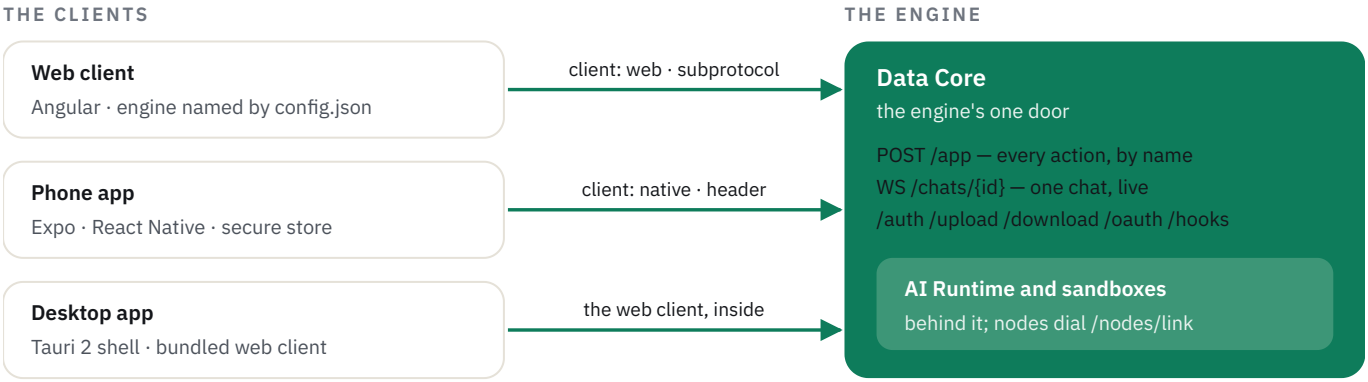


Figure 12.1 — Three clients, one API. Every client speaks only to the Data Core, across origins, with a session token it holds itself. The desktop app is not a fourth protocol: it is the web client in a window of its own. No client reaches the runtime, a node or the database.

	WEB CLIENT	PHONE APP	DESKTOP APP
Engine address	person's choice, else `assets/config.json`, else the page's origin	typed on the first screen; kept in the secure store	the shell answers `assets/config.json`
Token kept in	`localStorage` or `sessionStorage`, bound to its engine	the platform's secure store (the keychain on iOS)	as the web client, in the app's web view
Chat socket	subprotocol `decentai.bearer, <token>`	`Authorization` header on the handshake	as the web client
Speech	the computer's own voices and recognizer where local; else the server's	the phone's recognizer and voices, by default	as the web client
Notifications	web push (VAPID)	Expo's push service	no channel of its own
Scope	everything: chats, agents, settings, admin	chats, cards, schedules, stop-everything	everything, plus running the engine locally

> **I-12.1** No client is a security boundary. Every read a client is served is filtered by the Data Core inside the query, and every action it sends is resolved again from the session on the server; a modified client gains nothing an honest one lacks.

12.2 The web client

`decentai-web-client` is an Angular 20 single-page application. It is built once and the same files are served everywhere: by its own container (Caddy on port 80, plain HTTP), by any static host, or from inside the desktop app. What differs between deployments is one file read at run time, `assets/config.json`, which says `{"engine": "https://api.example.com"}`. The container writes it from `ENGINE_URL` as it starts; a static host has it written after the build. The file is fetched afresh on every start and never cached by the service worker, so changing it on the host moves every browser on its next load.

Before the application starts, the client resolves the engine in a fixed order: first an address the person entered, kept in the browser's storage; then the deployment's `config.json`; then the page's own origin, for a deployment whose reverse proxy serves the engine's paths beside the app's files. The last is only a guess, so the client probes it: `GET /auth/setup` must answer JSON carrying `setup_required`, which only an engine does. Otherwise the Connect screen asks.

The session token is the client's to hold. With *keep me signed in* it goes to `localStorage`; otherwise to `sessionStorage`, which ends with the tab; keeping it in one clears the other. The token is stored together with the engine that issued it and is dropped the moment the engine's address changes: one engine's session is never offered to another. A browser cannot set headers on a WebSocket handshake, and a token in the URL would be written into every log on the way, so the chat socket offers it as the second entry of its subprotocol list, after the name `decentai.bearer`; the Data Core answers with the name alone. Files are fetched with the token too and shown from a blob, since an `<img>` carries no header.

Because the engine is on another origin, the browser enforces the engine's `CORS_ALLOW_ORIGINS`: the Data Core answers preflights only for the origins named there, never a wildcard.

The chat page

A chat is shown as **turns**: a person's message and what came of it. The socket delivers events with sequence numbers. On reconnecting, the page replays what it missed from the Data Core's event log, holding live frames until the replay is applied so that no event is applied twice or out of order; if the log's tail has moved past where the page stopped, the page reads the chat again from its records instead of applying events over a hole.

A turn that did work has a **work block**: the assistant's plan as the runtime keeps it — each item with its status and what the trace proved for it (chapter 5) — and the calls made, grouped under their agents. A turn that only talks has no block; the answer is all there is. The plan is state, shown in place, not a second message.

Cards appear in the thread where the work waits. An approval card names the agent, the function, its level and a preview of the inputs of the one call it would release (chapter 6). A question card expects text, a choice, files, a credential, code to allow, or a skill to save. A card is answered once; an answered card cannot be answered again from the page.

Read-aloud says a reply in pieces of about 320 characters, so the first sentence is heard while the rest is prepared, and stops at 6,000 characters: a longer reply is a document, to be read. It uses the computer's own voices where it has them, or the organization's voice through the Data Core (`Settings:Speech`); which, and whether replies are said without being asked, is kept per device.

12.3 The phone app

`decentai-mobile` is an Expo (SDK 57) application in React Native 0.86 with Expo Router, for iOS and Android under one id. It is a remote control: it connects to any DecentAI server by address and keeps a person in the loop away from a desk. It covers chats (live replies, attachments, tables and charts, dictation and read-aloud), **Cards** (everything waiting for the person, in one list), schedules, background work, notification settings and the switch that stops everything of theirs. Agents, the marketplace, credentials and organization settings are set up in a browser.

The first screen takes a server address, an email and a password; an address without a scheme gets `https://`. Sign-in sends `client: "native"`, and the token comes back in the body. The address and the token are kept in the platform's secure store (the keychain on iOS); signing out deletes the token and keeps the address. Every request carries the token as a Bearer header. React Native, unlike a browser, lets a socket carry headers, so the chat socket presents the token as an `Authorization` header on the handshake rather than as a subprotocol; the Data Core accepts a cookie, the header or the subprotocol, in that order.

Push notifications go through Expo's push service. The app asks the operating system for permission, obtains an Expo push token for the project, and registers it with `Settings:Notifications:Subscribe`; signing out unsubscribes it. The Data Core needs no key to send through Expo (`EXPO_ACCESS_TOKEN` only raises rate limits); on Android, delivery also needs the project's Firebase credentials. Tapping a notification opens its chat. What a notification says follows the person's and the organization's choice: what happened, or only that something did (chapter 11).

Speech is done on the phone by default: its own recognizer, on-device only, and its own voices. A phone that cannot recognize on the device, or a person who turns it off, uses the server's speech instead. The phone must reach the server: `localhost` on a phone is the phone, so a server on a person's own computer needs a tunnel or a network address.

12.4 The desktop app

`decentai-desktop` is a Tauri 2 shell for Windows and macOS. It carries a released web client as files, untouched, and its own few screens. On first opening it asks one question, which it remembers: **connect to a server**, or **run DecentAI on this computer**. Either way DecentAI is the bundled web client in a window of the app's own, and the shell answers the one path that differs between computers itself: `assets/config.json`, with the server's address or `http://localhost:<port>`. The window drops any address the person chose inside the web client whenever the app's own choice changes.

The page's origin is `http://tauri.localhost` on Windows and `tauri://localhost` on macOS. No web page can claim those origins, which is why the engine's default `CORS_ALLOW_ORIGINS` lists them: a server that keeps them answers the app, one that removes them refuses it. The page is given none of the shell's native commands — those are granted to the app's own window only — so nothing rendered in a chat can stop or uninstall the engine. Sign-in windows that belong to the app or the engine stay in the app; every other link opens in the person's browser.

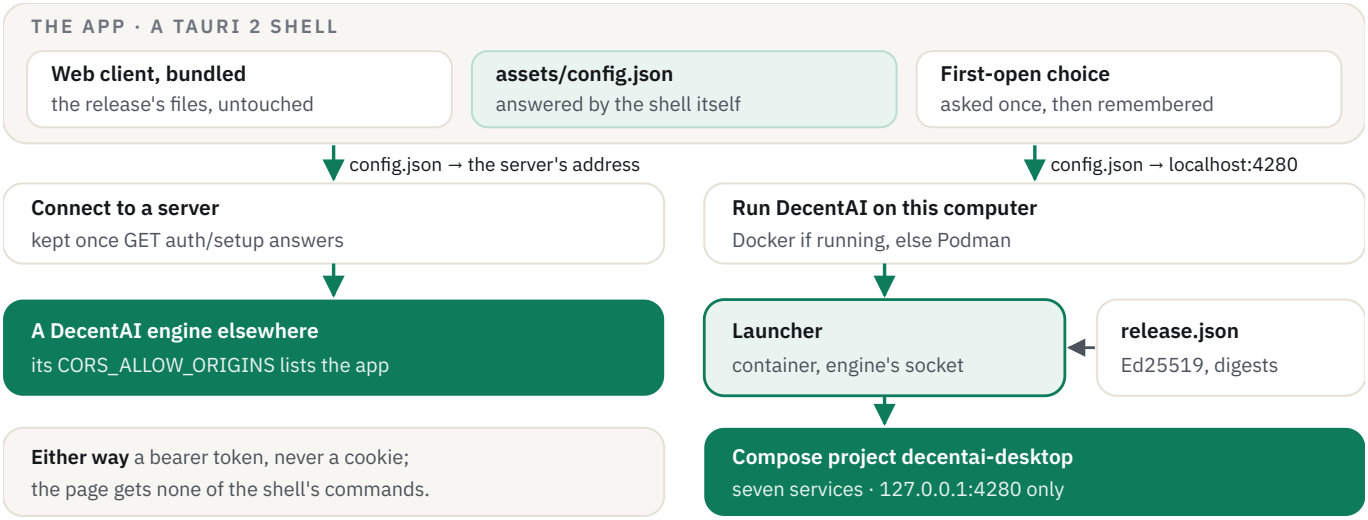


Figure 12.2 — The desktop app's two modes. One shell and one bundled web client; the only thing that differs is the engine `config.json` names. In the second mode the app installs nothing itself: a launcher container, handed the container engine's socket, starts the engine's released images, which it believes only when the release file's signature verifies.

Running DecentAI here

The engine is three images and a database; on a laptop they run under Docker or Podman. The app picks Docker where it is running and Podman otherwise, and keeps to the one an install was made on, because the data is there; where neither is installed, it offers to install Podman with the system's own installer. The app does no installing itself. It runs the **launcher**, a container built on a Docker command-line image with Python, handed the container engine's socket and a volume of its own (`decentai_desktop_launcher`). Because the launcher brings its own `docker compose`, it behaves the same on Windows and macOS, on Docker and Podman, and it is updated by pulling an image. The first administrator's password reaches it in the environment, never on the command line, and is kept nowhere.

The launcher starts its Compose stack as project `decentai-desktop` — a name nothing else on the machine uses, so its volumes never collide with a developer's own `decentai` stack. The services are the engine's own: `mongo`, `init`, `data-core`, `node-issuer`, `ai-runtime`, `sandbox` and `speech`, on the engine's three networks. Images are pulled by digest and never built. The keys are made once, by the engine's own generators run in the Data Core's image; a first run that stopped half way carries on with what it made, because keys made twice are a database nobody can read. Its settings say what one computer is: the Data Core bound to `127.0.0.1`, `CORS_ALLOW_ORIGINS` the app's two origins and nothing else, `SETUP_CODE=none` (nobody else can reach the address) and `ORG_SIGNUP=closed`.

A release of the app is a file, `release.json`, schema 2.0, with an Ed25519 signature beside it. It names by digest the engine's images (`data-core`, `ai-runtime`, `speech`), `mongo`, the launcher and the web client the installers carry, and the oldest launcher that understands its schema (`minimum_launcher`). The app reads it to learn which launcher to fetch, by digest; the launcher reads it again before it installs, and refuses a schema it does not know. Both believe it for its signature by one of the public keys the launcher ships with, and for nothing else. An unsigned file — a build of one's own — is installed only when explicitly allowed.

An update pulls every new image first, so an image that cannot be pulled stops it before anything is touched; then it copies the database, stops the services, runs the new seeder and starts the new release. If the stack does not come up healthy, the previous release is put back with the copy restored. The last three copies are kept.

> **I-12.2** The desktop app runs only images named by digest in a release file whose Ed25519 signature verifies against a key it ships with. Whoever could put a release file in front of an install could otherwise put their own software on the machine.

CHAPTER 13

Deployment

The platform is one image per process, configured entirely by environment. Moving between a laptop, a virtual machine and a cloud changes values, never code. The engine brings no TLS, no public hostname and no proxy: those are a deployment's, and the recipes in `decentai-deploy` supply them.

13.1 What is deployed

The engine is three images — the Data Core's, the AI Runtime's (also started as the Execution Sandbox and, from the Data Core's image, as the node issuer) and speech's — plus MongoDB 7. Started from the repository they are seven containers on three networks (chapter 2): `default`, which has a way out; `data`, internal, holding the database and the two processes that speak to it; and `sandbox`, internal, between the runtime and its sandboxes. Exactly one port is published: the Data Core's, plain HTTP, `4280` on the host by default. The web client is served apart, by its own container or a static host.

This shape is what makes the topologies below cheap. An execution node — one runtime with its sandboxes — *dials* the Data Core's `/nodes/link` over the same public address people use, and nothing dials a node (chapter 8). So moving agent code onto another machine opens no inbound port anywhere; it only needs that machine to reach the engine's HTTPS address.

13.2 Topologies

TOPOLOGY	RECIPE	WHAT RUNS WHERE	PUBLISHED
One computer, from source	the engine's setup script	the seven containers; the web client from its own checkout	`127.0.0.1:4280`
The desktop install	the desktop app's launcher	project `decentai-desktop`; the web client inside the app	`127.0.0.1:4280`
One server	`single-host`, `ROLE=all`	Caddy, web client, Data Core, Mongo, speech, runtime, sandbox, node issuer	Caddy: 80, 443
Core and a separate node	`single-host` with `ROLE=core`; `node`	core: Caddy, web client, Data Core, Mongo, speech. Node: runtime, sandbox, node issuer	core: 80, 443; node: nothing
AWS, two VMs	`cloud-vm/aws`, `layout = "two-vm"`	the core recipe on one EC2 VM, the node recipe on another	core: 80, 443; node: nothing
AWS, one VM	`cloud-vm/aws`, `layout = "one-vm"`	`single-host` with everything	80, 443

single-host needs a Linux host with Docker Engine and Compose 2.24 or later, two DNS names (`api.<domain>` and `app.<domain>`) and ports 80 and 443 open; Caddy obtains and renews certificates itself. The host needs no Python: the recipe's setup script pulls the Data Core's image at the pinned version and configures the deployment inside it, making every key with the engine's own generators the first time and keeping them every time after. It sets `PUBLIC_ENGINE_URL` and `PUBLIC_APP_URL` to the two names, `CORS_ALLOW_ORIGINS` to the app's name plus the desktop origins (`DESKTOP_ORIGINS`; empty refuses the app), and `FORWARDED_ALLOW_IPS` to Caddy's fixed address on its own network. Two names are the default because they keep the engine's origin apart from the app's, which is the case the engine is tested in; `LAYOUT=path` serves both under one name, the engine at `/api/`.

With `ROLE=core` the server runs the Data Core's side only and writes a **node bundle** once (`config/node-bundle.env`): the Data Core's public key and address, the node's sandbox secret and the node authority's private key. The bundle is carried to the node machine, where the **node** recipe splits it into the three settings files its services read and deletes it; it is then deleted on the core too, so the authority's private key exists on the node alone. The node publishes no port. It logs *Linked to the Data Core* once its link is up, and appears on the Data Core's Nodes page.

13.3 Two VMs on AWS

`cloud-vm/aws` is Terraform for the whole of it. It makes a VPC with one public subnet; two EC2 VMs (Ubuntu 24.04, Docker from Docker's packages, encrypted gp3 disks; `t3.large` each by default); an Elastic IP for the core; `api.` and `app.` records in Route 53 when a hosted zone is named (`route53_zone`); and daily snapshots of every disk at 03:00 UTC through Data Lifecycle Manager, the last seven kept (`snapshot_days`). The core's security group admits 80 and 443 (TCP, and UDP 443 for HTTP/3); the node's admits nothing. SSH is off unless a key pair and source ranges are given; both VMs are reached through Session Manager.

The VMs need no git and no access to GitHub: Terraform uploads the `single-host` and `node` recipes to a private S3 bucket, and each VM's first boot copies them from there and runs its recipe. Keys are made on the core. The node's share travels once through an encrypted SSM parameter (`/<name>/node-bundle`) and is then deleted from the core; the first-run setup code is published in another (`/<name>/setup-code`). Both VMs require IMDSv2 tokens with a hop limit of one, so no container can reach the VM's instance role through the metadata service. With `ecr_mirror = true` , the four images are pulled from ECR repositories in the account with the VMs' role, for images that are not yet public.

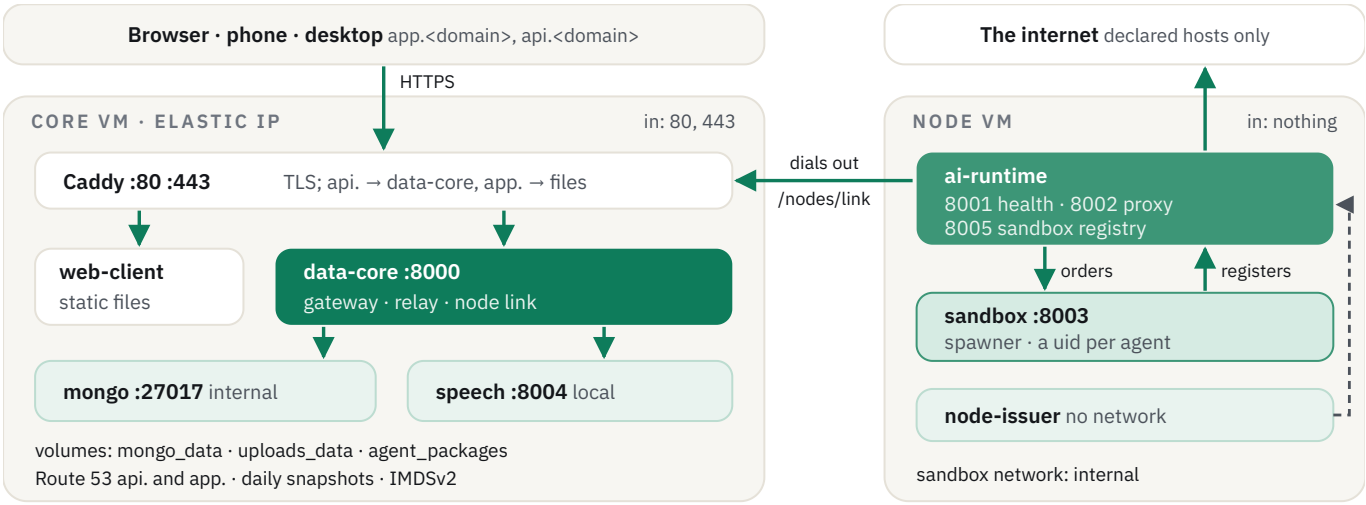


Figure 13.1 — The two-VM topology. People reach only Caddy on the core VM. The node VM accepts no connection: its runtime dials the Data Core's node link through Caddy over HTTPS, its sandbox registers with the runtime on 8005 and takes orders on 8003, and agents reach the internet only through the runtime's proxy on 8002. The node issuer, on no network, rewrites the node's credential into a shared volume (dashed). The node VM holds no database, no uploaded file and none of the Data Core's keys.

> **I-13.1** A deployment publishes one door: the Data Core's port, or the TLS terminator in front of it. The database is on an internal network reached by the Data Core and the seeder alone, and an execution node publishes nothing, because it dials the Data Core and is never dialed.

13.4 The desktop's local install

The desktop install (section 12.4) is the one-computer topology with the edges turned inward. The Data Core is published on `127.0.0.1` only, nothing stands in front of it (`FORWARDED_ALLOW_IPS=127.0.0.1`), only the app's two origins may call it, no setup code is asked, and no email is sent: invitations become temporary passwords shown once, and a forgotten password is reset by the launcher. Speech fetches no model until an administrator turns speech on, so a first run is not slowed by downloads. Reaching the install from a phone is a tunnel the person sets up.

13.5 The configuration surface

Every setting is an environment variable; the code reads the environment and nothing else. Compose reads the deployment's settings file, `config/deploy.env` , which is handed whole to the Data Core and the seeder. Two secrets are kept out of it because those two processes need neither: the node's sandbox secret, given to the runtime and its sandboxes, and the node authority's private key, given to the node issuer alone, each in a file of its own. The runtime is handed only the variables its service names; the database password, the encryption keys and the Data Core's private key never reach it. The settings that most change behaviour:

SETTING	DEFAULT	WHAT IT DECIDES
`PUBLIC_ENGINE_URL`	`http://localhost:4280`	the engine's address as browsers and providers reach it; the base of the OAuth redirect and of webhook addresses
`PUBLIC_APP_URL`	`http://localhost:4300`	the web client's address: links in invitations, resets and notifications; the only origin the OAuth popup reports to
`CORS_ALLOW_ORIGINS`	`localhost:4300`, `localhost:4200`, `http://tauri.localhost`, `tauri://localhost`	every origin the web client is served from, and nothing wider; never a wildcard, since any origin listed can act for a signed-in person
`FORWARDED_ALLOW_IPS`	`127.0.0.1`	whose `X-Forwarded-For` is believed: the TLS terminator only. Session records and the sign-in lockout (twenty failures in fifteen minutes per address) depend on it; never `*`
`TOKEN_SECRET_KEY`	required	signs session tokens. Missing, the template's `change-me`, or shorter than 32 characters: the process refuses to start
`SECRET_ENCRYPTION_KEYS`, `_ACTIVE`	required; highest version	versioned AES-256-GCM keys as `version:key`; every version the same width, since versions are ordered as text; mixed widths are refused
`FILE_STORAGE_PROVIDER`	`local`	`local`, `s3`, `azure_blob` or `gcs` for file bytes and agent packages; each file is read from the store its record names, so a store moved away from is still read
`SMTP_HOST`, `MAIL_FROM`	unset	outbound mail. Unset, nothing is sent: people are added with a temporary password shown once
`VAPID_PUBLIC_KEY`, `_PRIVATE_KEY`	unset	the web-push pair. Unset, nothing is pushed to browsers; email is used where configured
`SETUP_CODE`, `ORG_SIGNUP`	empty; `closed`	empty makes a first-run code and writes it to the log, so whoever finds the address first cannot claim it; sign-up stays shut without mail
`DECENTAI_VERSION`	unset (`local`)	the release whose images are pulled; unset, the images are built from the checkout

The keys are generated, never chosen: the Data Core's RS256 service pair, the node authority's pair, the encryption keys and the token secret each have a generator, and none writes its output to disk. Rotation of the encryption keys is non-destructive: the generator appends a version, new writes use it, and a re-encryption tool rewrites stored values until the old version can be dropped. Two things are deliberately *not* settings: a ceiling on trust levels and a turn budget. Trust is a chat's, bounded by what its person may do; budgets are per chat.

> **I-13.2** A database is readable only with the keys it was written with. Setup and update scripts make each key once and keep it thereafter; they add a key a newer version needs and never replace one that exists.

13.6 Volumes and backups

`mongo_data`	the database: identities, chats, records, encrypted values. Back up.
`uploads_data`	file bytes (with the `local` store). Back up.
`agent_packages`	approved agent packages — the only copy the platform controls. Back up.
`runtime_agents`, `sandbox_data`	caches of approved code by digest, environments, and the monitoring record; rebuilt from the packages when lost.
`speech_models`, `node_credential`	models fetched again on demand; a credential rewritten every few minutes. Not worth a backup.
the settings	without `SECRET_ENCRYPTION_KEYS` every stored credential is unreadable, and without the authority's private key the node authority is lost.

`single-host`'s backup script writes the three things that together are a deployment: the settings with their keys, a `mongodump` archive taken while the database runs, and the file and package volumes. On AWS the daily disk snapshots come in addition, not instead.

13.7 Updates

A release is a version tag on a commit whose tests passed; the release workflow refuses any other. It builds the three images for Intel and ARM, publishes them to `ghcr.io/decentai-io` under the version and `latest`, and makes a GitHub release that names each image's digest. The web client is released by its own repository, as an image and as a static archive. Recipes pin both versions separately and pull; they never build.

An update seeds first: the new seeder (`docker compose run --rm init`) applies the schema and grants new baseline actions while the old code still serves, then the services are rolled. A newer seed adds collections and actions and drops no data, so older code runs against a newer database and going back is naming the previous version. Two things every version removes as it starts: an index its schema does not declare, and, from every policy, an action its catalog no longer has. A node that speaks a different protocol from its Data Core is refused by name, so a core and its nodes are updated together.

CHAPTER 14

Verification

Each property this paper states is meant to be checked, not believed. The checks are automated test suites that use the platform the way a person, an agent or an attacker would; a hostile agent run in the deployed containers; and tests that hold the documentation to the code. This chapter says what runs, how, and what the long paper reported.

14.1 The suites

The counts below are test functions in each repository at the time of writing; parametrization makes the executed count higher. They grow with every fix: the long paper's counts of 15 October 2026 (section 14.5) are smaller.

SUITE	TESTS	RUNS AGAINST	EXERCISES
AI Runtime	1,233	an in-memory simulator of the Data Core's services; no database	the assistant loop, executor gates, approvals, clock, stops, proxy, evidence, the documentation
Data Core	1,125	MongoDB 7	identity, policy invariants, the delegation fence, data layer and encryption, OAuth, webhooks, nodes, audit
Spanning	190	both, a real runtime process	agent install and manifest, scope enforcement, the chat contract, the full stack
Confinement, live	97 of the runtime's	the runtime's and two sandboxes' containers on Linux	a hostile agent's escapes, and the spawner across containers
Speech	20	the speech image; 3 with the real models	the audio protocol, the model catalogue's digests
Web client	about 670 cases	Karma and Jasmine, headless Chrome	services, chat state and connection, cards, components
Desktop launcher	95	pytest	release verification, installation, updates and roll-back
Desktop shell	10	`cargo test`	the first-open choice, the release file, the window

Continuous integration runs four jobs on every pull request: the runtime suite with the documentation checks; the Data Core and spanning suites against a `mongo:7` service, one after the other because they share the test database; the confinement suites in containers; and speech, once without its models and once with them. The phone app has no unit suite; it is held to the TypeScript compiler and Expo's linter. The deploy repository is held to `docker compose config`, `terraform fmt` and `validate`, shellcheck and PSScriptAnalyzer, none of which needs a cloud account.

14.2 A scripted model

Runtime tests run real agent code in real worker processes with a **scripted model**: a stand-in that returns replies written in advance. A test must give the same result on every run, and a real model does not; and the claims under test are about what the platform does with *whatever* the model proposes. The simulator answers the Data Core's services contract in memory; it is also how an agent author tests an agent alone. The spanning suite closes the gap the simulator leaves: in its full-stack test a person's socket reaches the Data Core, the Data Core attaches a real runtime process serving the same code the image serves, and the runtime calls back over `/app` under the chat's delegation. Nothing is stubbed but the model.

14.3 An agent written to escape

The live suites install a fixture agent that is hostile on purpose: each function attempts one escape — read the runtime's settings, write into its own package, signal the runtime, connect past the proxy, resolve a name directly, open an unused socket family, `ptrace`, create a namespace — and the test passes only if the attempt is refused. Beside each refusal a positive control checks that ordinary work still functions, because a sandbox that refuses everything would pass the first kind of test and fail the second. A second hostile package attacks at build time, from a local package index.

> **I-14.1** A green confinement run has run the hostile agent. The live suites skip everywhere but the container stack; the CI job fails when either passed nothing, so a run in which every test was skipped for want of a kernel feature cannot be reported as a pass.

14.4 Documentation held to the code

The documentation names the file that implements each mechanism, and this paper is distilled from it. Five tests keep the two in step: every relative link leads to a page and to a heading on it; every repository path a page names exists; every setting the configuration reference names is read by some code; every setting the example settings files offer is in the configuration reference; and the published list of actions is exactly what its generator writes from the Data Core's catalog today. These tests check what a page *points at*. Whether a sentence is true remains a reader's to verify.

14.5 The reported evaluation

The long paper (October 2026) reports these suites, as test functions, from runs on 15 October 2026. Its "Backend" is the Data Core.

SUITE	TESTS	EXERCISES
Runtime	1,031	assistant loop, executor gates, approvals, clock, stops, proxy, evidence, documentation
Backend	924	identity, policy invariants, delegation fence, data layer and encryption, OAuth, webhooks, audit
Cross-component	180	approval-to-package path; end-to-end chat across backend and runtime
Example agent	24	an agent run as its author would
Total	**2,159**	
of which sandbox, live	83	hostile agent in the deployed two-container topology

On the main branch on that date (Python 3.11, MongoDB 7) every suite passed with no failures: the runtime with 1,056 passed and 85 skipped (the 83 live sandbox tests, which need the Linux image and were run separately, and two others); the backend with 969 passed and 6 skipped; cross-component with 216 passed and 3 skipped; the example agent with 24 passed. The live suites were then run in the deployed topology — the runtime's image as the runtime, and again as the agents' container beside it on an internal network, with `NET_ADMIN` for the egress rule — on Linux 6.18 (WSL2), which offers every Landlock feature the sandbox uses. All 83 passed, none skipped: 58 confinement tests and 25 of the spawner across the two containers, in 88 seconds.

The same month, the whole code base was reviewed line by line. Every case found to make the code fail, hang or answer wrongly was fixed together with a test that repeats it; in the runtime these regression tests are grouped by area — around stops (26 tests), the executor (17), contracts and the SDK (31), the clock (8), session restarts (6), concurrent stores (8), reasoning budgets (14), and services and connectors (24). Design questions the review raised and did not settle were published instead (chapter 15).

14.6 What is not evaluated

- **Model behaviour.** No test calls a real model. The tests show that a proposal cannot pass the gates; they do not measure whether a model resists injected text or chooses well.
- **Performance and scale.** No latency or throughput overhead is reported, nor behaviour with many organizations, chats or concurrent schedule fires.
- **The kernel.** Confinement results hold on the kernel tested. A kernel flaw is a way past every layer, and no test rules one out.
- **The clients.** The web client's suite exists but was not part of the reported evaluation; the phone app has none.

CHAPTER 15

Limits and open questions

What the platform does not do or guarantee, in one place. Each limit is **chosen** (a trade-off made on purpose), **open** (raised by the October 2026 review and not yet decided) or **inherited** (from what the platform runs on).

15.1 What is not guaranteed

AREA	LIMIT	KIND
The model	The platform does not make the model correct. Injected text still steers whatever runs without a card: every granted function at or below the chat's trust level, which at level 3 is every granted function. There is deliberately no administrator ceiling on the level.	chosen
Plans	A plan item counts as verified when it has *some* evidence: one unrelated successful call can attach to every active item, and the model chooses which reference it cites (RSN-10).	open
Approved agents	During a call an agent holds the decrypted credential and may talk to its declared hosts; the proxy sees host and byte counts, never content. Warm workers could keep a credential beyond the call. Review before approval is the control.	chosen
Declared hosts	A declared host is in effect a declared address: once connected, an agent can ask that address for another site it serves (SBX-6).	open
Agents together	Two colluding agents can pass bytes through `/tmp` or `/dev/shm` (SBX-9), or through an abstract socket before Linux 6.12. One worker per approved agent per organization serves everyone granted it, so concurrent colleagues share its process.	open; inherited; chosen
The kernel	All containers share it; some locks need a recent kernel or rights the operator grants, and the platform runs and says so where they are missing. The system-call filter exists on x86-64 and arm64 only, and switching it off switches it off for every agent.	inherited
Resources	Memory is a shared pool sampled once a second; disk use is counted but not capped. Tenants share the agents' container, and an agent writing enough to its error output can push others' connection records out of the bounded log (LIB-7).	chosen; open
A compromised node	It can act for the people whose delegations it holds — within the 41-action fence, for up to an hour, and without opening cards — because a scheduled chat must act while its person is away. It cannot decide a card, read the database or open a sealed value.	chosen
The audit trail	Append-only by the code's shape, not by mathematics: there is no hash chain, so someone holding the database password could alter it undetectably. A call's inputs are recorded in outline as the model proposed them, so a password typed into a chat can reach it (EXE-11).	chosen; open
Secrets in content	Results are validated for shape, not scanned for secrets: a credential an agent returns reaches the model and the trail. File bytes are not sealed by the platform; they rest on its volume or in a bucket the provider encrypts.	chosen
Webhooks	Only DecentAI's own signature scheme is verified. An outbound address is checked by name and connected to by name, leaving a DNS-rebinding window; an event a chat could not receive is not retried.	open
Failover	A chat whose node stops answering waits for its lease to lapse (60 seconds) before it moves. A call that was running there is reported as unknown and never run again by itself.	chosen
The Data Core	It is the component trusted with everything: a compromised Data Core is a compromised deployment.	chosen

15.2 What is not built yet

No release has been published at the time of writing. The `decentai-deploy` recipes run released images and pull rather than build, so until a release exists they run images of one's own (or the ECR mirror, on AWS), and the engine's own deploying guide is the way onto a server. Recipes for Azure, Google Cloud, Kubernetes and marketplaces are planned and absent. The desktop install listens on `127.0.0.1` only and sends no email; it does not carry over installs from the earlier monolithic app. The phone app configures nothing but its own use; agents, credentials and settings need a browser.

15.3 Open questions

The October 2026 review raised design questions it did not settle: choices, not defects, where the code does what it does on purpose or by omission and either answer may be right. They are published with the code as a list of open questions, each naming the function it concerns and whether it was reproduced ("run") or found by reading. The long paper, written mid-month, counts 137; the list holds 130 at the time of writing, as rows are deleted when decided. A sample of the ones a deployer should know:

ID	QUESTION
EXE-12	Every string in a call's inputs is treated as a reference the function may read, whether or not the input is marked as one; in the simulator one agent read another's record when the reference was in its inputs.
EXE-14	There is no ceiling on a function's `timeout_seconds`, on how often a function asks the model, or on how many cards one call opens.
ASST-14	Any object with an `"action"` key inside a prose reply is run as an action, so a quoted example can be executed.
SBX-7	A credential field holding `.com` is lent as a host pattern, and a leading dot means every host under it.
LIB-5	A declared dependency may still be a direct URL, a `git+` address or a path, which the builder fetches.
EDGE-9	Tool arguments that do not parse become no arguments, silently, in every model connector.

Publishing these is deliberate. A security design that lists only what works asks to be taken on faith; one that lists where it is undecided lets a reader weigh the rest.

APPENDIX

Glossary

The words this paper uses for one exact thing.

Action — one operation the gateway dispatches, named `Domain:Controller:action`; also the name a policy grants. Nothing is permitted by default.

Agent — an approved program: a folder in a git repository holding a manifest and the code that does what it declares, installed as one exact commit.

AI Runtime — the process that runs the assistant of each chat, the executor every call passes, and the egress proxy. It has no database and no key to one.

Audit trail — the append-only record of what the platform allowed: calls, approvals, credential reads, installs, grants, stops and gateway changes.

Authority — a public key or an issuer's key set that may vouch for execution nodes, for one organization or shared. Nodes are admitted by authority, never one by one.

Beat — one turn of the assistant's cycle: one model call, one proposed action.

Card — something in a chat that waits for a person: an approval of one exact call, a question, code to allow, a login.

Credential slot — one resource of one installed agent that a credential is granted to, named `agt_<ref>__<resource id>`. The agent receives the value for one call.

Data Core — the engine's one door: identity, policy, the vault, the data layer and the relay. The only part that reaches the database.

Delegation — the short-lived credential the Data Core mints per chat, bound to one node, under which the runtime calls back as the person within a fixed list of 41 actions.

Evidence — what the platform, not the model, shows beside a reply: tables, charts and files rendered from stored results that the trace proves exist.

Execution node — one AI Runtime and its Execution Sandboxes, deployed together. It dials the Data Core's node link and is never dialed.

Execution Sandbox — a container where agents' workers run and nothing else; it registers with its runtime and reaches nothing but it.

Function — one operation an agent's manifest declares, with input and output schemas and a permission level.

Gateway — `POST /app`, the one route every action goes through and where permission is checked.

Grant — permission to use something: a policy allowing an action; an agent grant to a person or group; a secret grant lending a credential to an agent's slot; a chat's granted functions.

Launcher — the desktop app's container, handed the Docker or Podman socket, that installs, updates and removes the engine's containers on one computer.

Lease — which node holds an active chat, numbered and renewed by heartbeat; a call carrying an old number is refused.

Manifest — an agent's contract with the platform: its functions and levels, the records and files it keeps, the credential it needs, the hosts it reaches and its dependencies.

Permission level — 0 to 3 on a function: a read, an ordinary change, a change with wider reach, an action that leaves the platform or cannot be undone.

Plan item — one entry in the assistant's plan, kept by the platform with its status and the stored references offered as its proof.

Proxy — the runtime's egress proxy on port 8002: a worker's only way to the internet, open to the hosts its agent declared.

Record — what an agent keeps for a person, in the shape its manifest declared, with plaintext keys and encrypted values.

Release file — the desktop app's `release.json`: image digests and the minimum launcher, believed only for its Ed25519 signature.

Skill — a short instruction an organization wrote for the assistant, pulled into a chat when the task calls for it.

Spawner — the platform's one process in an Execution Sandbox, which starts, builds and ends workers when the runtime asks.

Trace — what the assistant actually ran in a chat; evidence and plan proofs are checked against it.

Trust level — 0 to 3 on a chat, chosen by its person: a function at or below it runs unasked; above it, it waits for a card.

Worker — the process one approved agent's code runs in, as a user of its own, fenced to its own files.